

Таблица 1 – IP адреса

Устройство	Интерфейс	Адрес	Шлюз	Префикс	Маска
ISP	eth2 (Bridge)	DHCP	DHCP	24	255.255.255.0
	eth1 (В сторону HQ)	172.16.4.1		28	255.255.255.240
	eth0 (В сторону BR)	172.16.5.1		28	255.255.255.240
HQ-SRV	ens18				
	ens18.100	192.168.1.2	192.168.1.1	27	255.255.255.224
HQ-RTR	eth0(K ISP)	172.16.4.2	172.16.4.1	28	255.255.255.240
	eth1				
	eth1.100	192.168.1.1		27	255.255.255.224
	eth1.200	192.168.2.1		27	255.255.255.224
	gre1	10.10.10.1		30	255.255.255.252
BR-RTR	eth1 (K ISP)	172.16.5.2	172.16.5.1	28	255.255.255.240
	eth0	192.168.4.1		28	255.255.255.240
	gre1	10.10.10.2		30	255.255.255.252
BR-SRV	ens18	192.168.4.2	192.168.4.1	28	255.255.255.240
HQ-CLI	ens18				
	ens18.200	DHCP	DHCP	DHCP	255.255.255.240

Если найдете ошибку в мануале, либо у вас будут вопросы, пишите в тг [@Arugva](#), по возможности помогу.

Светло-серым цветом в данном мануале я буду помечать команды, которые требуется выполнить, а темно-серым пути к файлам конфигурации. Это сделано для удобства читателя, если вам не хочется разбираться что и как мы делаем, а лишь быстро выполнить работу и пойти отдыхать:)

Вещи, которые делать необязательно, но они могут облегчить дальнейшее решение экзамена:

0.1. Отключение IPv6 на всех устройствах

По дефолту айпив6 включен и активен на всех виртуальных машинах, и, хоть вероятность небольшая, но он может вам помешать и устройства будут пытаться достучаться друг до друга именно по нему, а не v4. Именно так у меня было, когда я первый раз решал демоэкзамен, поэтому я решил перестраховаться и отключить его. Делается путем добавления в конфиг /etc/sysctl.conf строк

- net.ipv6.conf.all.disable_ipv6=1
- net.ipv6.conf.default.disable_ipv6=1

```
#
net.ipv6.conf.all.disable_ipv6=1
net.ipv6.conf.default.disable_ipv6=1
```

И, после добавления этих строк в конфиг применяем правила командой `sysctl -p`.

0.2. Добавление рекурсии ДНС

Рекурсия дает возможность нашему ДНС серверу отправлять запросы другим DNS-серверам, например, гугл. То есть, нам не придется на каждой виртуалке добавлять дополнительную строку `nameserver 8.8.8.8` в `resolv.conf`. Что бы добиться рекурсии, на HQ-SRV открываем конфиг `/etc/bind/options.conf` и добавляем строки

```
recursion yes;
```

```
allow-recursion { any; };
```

```
dnssec-validation no;
```

В конечном итоге этот конфиг должен выглядеть примерно так

```
options {
    version "unknown";
    directory "/etc/bind/zone";
    dump-file "/var/run/named/named_dump.db";
    statistics-file "/var/run/named/named.stats";
    recursing-file "/var/run/named/named.recursing";
    secroots-file "/var/run/named/named.secroots";

    pid-file none;
    listen-on { any; };
    listen-on-v6 { none; };
    recursion yes;
    allow-recursion { any; };
    dnssec-validation no;
    forward only;
    forwarders { 8.8.8.8; 192.168.4.2; };
};
```

Проверяем на любом клиенте, что в `resolv.conf` не указан сторонний сервер и у нас пингуется `ya.ru`

```
[root@br-srv sites-available]# cat /etc/resolv.conf
nameserver 192.168.1.2
search au-team.irpo
[root@br-srv sites-available]# ping ya.ru
PING ya.ru (77.88.55.242) 56(84) bytes of data.
64 bytes from ya.ru (77.88.55.242): icmp_seq=1 ttl=53 time=38.5 ms
64 bytes from ya.ru (77.88.55.242): icmp_seq=2 ttl=53 time=50.0 ms
^C
--- ya.ru ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1077ms
rtt min/avg/max/mdev = 38.531/44.247/49.964/5.716 ms
```

0.3. Исправление ошибки с бинарниками `/usr/sbin/` на HQ-CLI

На убогой операционной системе Alt Workstation войти из-под рута при запуске ОС почему-то не получается (по крайней мере у меня на версии 10.1 никак не получилось), поэтому, заходим из под дефолтного юзера и в терминале вводим `su – root`, после этого бинарники `/usr/sbin/` станут нам доступны.

0.4. Краткий гайд на VIM

Текстовый редактор `vim` может работать в нескольких режимах. Что бы переключить режим, нам нужно перейти в обычный режим, для этого нажимаем `Esc` (при открытии `Vim` мы сразу оказываемся в обычном режиме). Переход в разные режимы используются следующие клавиши:

- Режим вставки – `I`;
- Визуальный режим (выделение) – `V`;
- Командная строка – `:`

Основные команды командой строки:

`:w` – сохранить;

`:q` – выйти;

`:x` – сохранить и выйти;

`:q!` – выйти без сохранения;

`:set number` – включить номера строк.

Модуль 1

1. Произведите базовую настройку устройств

1.1. Настройка имен устройств согласно топологии с использованием полного доменного имени

Для настройки имен устройств используется утилита `hostnamectl` с использованием команды `set-hostname`.

Рассмотрим команду на примере настройки имени `BR-SRV`. Нам требуется выдать ему имя `br-srv.au-team.irpo`. Для начала посмотрим, какое имя у него сейчас, для этого введем команду `hostnamectl`

```

[root@mwie8yczkor.jz ~]# hostnamectl
Static hostname: mwie8yczkor.jz
Icon name: computer-vm
Chassis: vm
Machine ID: 581822a8d758df7c7704f7d86906f27e
Boot ID: 3335dff5224c4719b730d603a4c131ad
Virtualization: qemu
Operating System: ALT Server 10.2 (Mendeleevium)
CPE OS Name: cpe:/o:alt:server:10.2
Kernel: Linux 6.1.79-un-def-alt1
Architecture: x86_64
Hardware Vendor: QEMU
Hardware Model: Standard PC _i440FX + PIIX, 1996_

```

Видно, что у него используется имя, которое система сама выдала при установке (если мы там не ввели свое имя).

Теперь сменим его командой `hostnamectl set-hostname br-srv.au-team.irpo`, что соответствует топологии и сразу проверим, применилось ли наше имя

```

[root@mwie8yczkor.jz ~]# hostnamectl set-hostname br-srv.au-team.irpo
[root@mwie8yczkor.jz ~]# hostnamectl
Static hostname: br-srv.au-team.irpo
Icon name: computer-vm
Chassis: vm
Machine ID: 581822a8d758df7c7704f7d86906f27e
Boot ID: 3335dff5224c4719b730d603a4c131ad
Virtualization: qemu
Operating System: ALT Server 10.2 (Mendeleevium)
CPE OS Name: cpe:/o:alt:server:10.2
Kernel: Linux 6.1.79-un-def-alt1
Architecture: x86_64
Hardware Vendor: QEMU
Hardware Model: Standard PC _i440FX + PIIX, 1996_

```

Имя применилось, однако в терминале по-прежнему пишет наше старое имя, чтобы это исправить можно либо перезагрузить виртуальную машину, либо перезагрузить оболочку `bash` командой `exec bash`

```

[root@mwie8yczkor.jz ~]# exec bash
[root@br-srv ~]#

```

Теперь у нас отображается правильное имя.

По такой аналогии настраиваем все устройства в нашей сети.

1.2. Настройка конфигурации IPv4 согласно RFC1918.

Документ RFC1918 гласит о том, что в локальных сетях у нас должны использоваться адреса 10.0.0.0-10.255.255.255, 172.16.0.0-172.31.255.255 и 192.168.0.0-192.168.255.255 и именно их мы будем использовать при конфигурации сети.

Настройка сети на ISP.

Перед началом нам требуется узнать названия интерфейсов, которые у нас используются, для этого используем команду `ip a`.


```

root@isp:/home/user# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether bc:24:11:4a:c2:ca brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether bc:24:11:b6:fe:ed brd ff:ff:ff:ff:ff:ff
4: eth2: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether bc:24:11:c8:68:2f brd ff:ff:ff:ff:ff:ff

```

В моем случае интерфейсы называются eth0 (в сторону BR) и eth1(в сторону HQ), в вашем случае интерфейсы могут называться иначе (см. последовательность сетевых интерфейсов в настройках виртуалки в proxmox).

*eth2 – В моем случае интерфейс, который смотрит во внешнюю сеть (интернет), получает адрес по dhcp от вашего роутера.

Согласно моей таблице адресов, интерфейс eth0 имеет адрес 172.16.5.1/28, а eth1 имеет адрес 172.16.4.1/28.

Для настройки сети открываем конфиг `/etc/network/interfaces` и заполняем следующим образом:

```

# The loopback network interface
auto lo
iface lo inet loopback

auto eth2
iface eth2 inet dhcp

auto eth0
iface eth0 inet static
    address 172.16.5.1/28

auto eth1
iface eth1 inet static
    address 172.16.4.1/28

```

После чего перезагружаем сеть и смотрим, применились ли наши настройки:

```

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether bc:24:11:4a:c2:ca brd ff:ff:ff:ff:ff:ff
    inet 172.16.5.1/28 brd 172.16.5.15 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe4a:c2ca/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether bc:24:11:b6:fe:ed brd ff:ff:ff:ff:ff:ff
    inet 172.16.4.1/28 brd 172.16.4.15 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:feb6:feed/64 scope link
        valid_lft forever preferred_lft forever
4: eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether bc:24:11:c8:68:2f brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.25/24 brd 192.168.0.255 scope global eth2
        valid_lft forever preferred_lft forever
    inet6 2a01:540:99e:6200:be24:11ff:fec8:682f/64 scope global mngtmpaddr dynamic
        valid_lft 86251sec preferred_lft 86251sec
    inet6 fe80::be24:11ff:fec8:682f/64 scope link
        valid_lft forever preferred_lft forever

```

Видно, что конфигурация IPv4 в порядке, значит можно двигаться далее.

Настройка сети на BR-RTR.

Все изменения конфигурации сети будут также проводится в конфиге `/etc/network/interfaces`. Открываем конфиг и вводим адреса, которые указаны в таблице адресов

```
auto lo
iface lo inet loopback

auto eth1
iface eth1 inet static
    address 172.16.5.2/28
    gateway 172.16.5.1

auto eth0
iface eth0 inet static
    address 192.16.4.1/28
```

По заданию, сеть в сторону BR-SRV должна вмещать не более 16 адресов, это у нас может обеспечить маска подсети 28.

После настройки перезагружаем сеть командой `systemctl restart networking` и смотрим, применились ли наши настройки

```
root@br-rtr:/home/user# systemctl restart networking
root@br-rtr:/home/user# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether bc:24:11:38:32:b0 brd ff:ff:ff:ff:ff:ff
    inet 172.16.5.2/28 brd 172.16.5.15 scope global eth0
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether bc:24:11:22:fe:e4 brd ff:ff:ff:ff:ff:ff
    inet 192.16.4.1/28 brd 192.16.4.15 scope global eth1
        valid_lft forever preferred_lft forever
```

Настройка сети на BR-SRV

На серверах у нас используется ОС Alt server, на которой предустановлен etcnet, а не networkd (зачем, если все равно используется systemd – непонятно), к которому все так привыкли, поэтому настройка сети производится несколько иначе.

Для настройки открываем конфиг по пути `/etc/net/ifaces/*имя интерфейса*/options` и приводим к следующему виду:

```
BOOTPROTO=static
ONBOOT=yes
TYPE=eth
DISABLED=no
NM_CONTROLLED=no
```

Здесь:

- BOOTPROTO=static – указываем, что интерфейс имеет статический адрес;
- ONBOOT=yes – включать при старте системы;
- TYPE=eth – указываем тип ethernet;
- DISABLES=no – указываем, что интерфейс должен быть включен (самая подлая строка, которая по умолчанию имеет значение yes);
- NM_CONTROLLED=no – отключаем управление интерфейсом через NetworkManager.

После этого, все в той же директории нашего интерфейса создаем и открываем файл `ipv4address` и вводим адрес `192.168.4.2/28`, а также создаем и открываем файл `ipv4route` и вводим адрес `BR-RTR (192.168.4.1)`. После этого перезагружаем сеть командой `systemctl restart network` и смотрим на результат командой `ip a`

```
[root@br-srv ~]# systemctl -p
[root@br-srv ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:15:31:58 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    inet 192.168.4.2/28 brd 192.168.4.15 scope global ens18
        valid_lft forever preferred_lft forever
```

Настройка сети на HQ-RTR

На HQ-RTR у нас должны быть настроены VLAN (Да, на маршрутизаторе VLAN, поскольку у нас нету виртуального коммутатора 😊), а значит его настройка будет чуть отличаться от остальных.

Открываем конфигурационный файл `/etc/network/interfaces` и приводим к следующему виду:

```
auto eth0
iface eth0 inet static
    address 172.16.4.2/28
    gateway 172.16.4.1

auto eth1
iface eth1 inet manual

auto eth1.100
iface eth1.100 inet static
    address 192.168.1.1/27

auto eth1.200
iface eth1.200 inet static
    address 192.168.2.1/28
```

Перезагружаем сеть и смотрим, что у нас появились интерфейсы eth1.100 и eth1.200

```
root@hq-rtr:/home/user# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether bc:24:11:ce:4f:08 brd ff:ff:ff:ff:ff:ff
    inet 172.16.4.2/28 brd 172.16.4.15 scope global eth0
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether bc:24:11:fd:91:c6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.1/27 brd 192.168.1.31 scope global eth1.100
        valid_lft forever preferred_lft forever
6: eth1.100@eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:fd:91:c6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.1/27 brd 192.168.1.31 scope global eth1.100
        valid_lft forever preferred_lft forever
7: eth1.200@eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:fd:91:c6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.1/28 brd 192.168.2.15 scope global eth1.200
        valid_lft forever preferred_lft forever
```

Интерфейсы появились, двигаемся дальше.

Настройка сети на HQ-SRV

Поскольку HQ-SRV настраивается через etcsnet и является частью VLAN, настройка на нем немного отличается.

Для начала создадим директорию /etc/net/ifaces/ens18.100/ командой `mkdir -p /etc/net/ifaces/ens18.100`

***У вас имя интерфейса может отличаться, смотрите ip a**

В ней создаем файл options и заполняем следующим образом:

```
HOST=ens18
VID=100
TYPE=vlan
ONBOOT=yes
BOOTPROTO=static
DISABLED=no
```

После чего командой `systemctl restart network` перезапускаем сеть и смотрим, появился ли у нас интерфейс ens18.100

```
[root@hq-srv ifaces]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:74:50:d5 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
4: ens18.100@ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:74:50:d5 brd ff:ff:ff:ff:ff:ff
```

Интерфейс есть, значит можем назначать ему адрес и шлюз. Для этого в директории интерфейса ens18.100 создаем файлы ipv4address и ipv4route, вписываем в

ipv4address 192.168.1.2/27, а в ipv4route 192.168.1.1, перезагружаем сеть и смотрим, появился ли наш адрес

```
[root@hq-srv ifaces]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:74:50:d5 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
6: ens18.100@ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether bc:24:11:74:50:d5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.2/27 scope global ens18.100
        valid_lft forever preferred_lft forever
```

Настройка сети на HQ-CLI

HQ-CLI должен получать всю конфигурацию по DHCP, который мы будем настраивать позже, сейчас нам требуется лишь добавить интерфейс с VLAN. Делается это почти как настройка на HQ-SRV.

Создаем директорию ens18.200 командой `mkdir -p /etc/net/ifaces/ens18.200/`, в ней создаем файл `options` и заполняем следующим образом:

```
HOST=ens18
VID=200
TYPE=vlan
ONBOOT=yes
DISABLED=no
BOOTPROTO=dhcp
```

Также, не забываем в конфиге options хостового интерфейса ens18 изменить значение параметра DISABLED с yes на no.

*Также, на HQ-CLI отличается применение правил ядра (`sysctl -p`), поскольку там нет бинарника файла `sysctl` (даже в `/usr/sbin/`). Что бы применить изменения, внесенные в `sysctl.conf` нам потребуется перезагрузить юнит `systemd-sysctl` командой `systemctl restart systemd-sysctl`

2. Настройте доступ к сети Интернет, на маршрутизаторе ISP

Вообще не понимаю смысл этого задания, поскольку ISP у нас подключен мостом к хосту и проблем настройкой интернета возникнуть не должно абсолютно никаких, но да ладно.

Мостовой интерфейс на ISP (в моем случае eth2) у нас настроен на DHCP, а потом после перезагрузки сетевой конфигурации после настройки сети должен был получить адрес и, соответственно, занять интернет. Если он у вас по какой-то причине не получил адрес, выполните команду `dhclient` (или же `dhcpcd`, в зависимости от версии ОС и установленных пакетов)

Также, сразу включим форвардинг пакетов на уровне ядра. Для этого открываем конфиг `/etc/sysctl.conf` и допишем строку `net.ipv4.ip_forward=1`, после чего применим наши правила командой `sysctl -p`

```
root@isp:/home/user# sysctl -p
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv4.ip_forward = 1
```

Проверим доступ в интернет, выполнив пинг серверов google

```
root@isp:/home/user# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=106 time=47.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=106 time=47.5 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=106 time=47.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=106 time=47.8 ms
```

Также проверим работу нашего DNS, попробовав пингануть ya.ru

```
root@isp:/home/user# ping ya.ru
PING ya.ru (5.255.255.242) 56(84) bytes of data.
64 bytes from ya.ru (5.255.255.242): icmp_seq=1 ttl=247 time=31.3 ms
64 bytes from ya.ru (5.255.255.242): icmp_seq=2 ttl=247 time=32.4 ms
64 bytes from ya.ru (5.255.255.242): icmp_seq=3 ttl=247 time=38.4 ms
```

3. Создайте локальные учетные записи на серверах HQ-SRV и BR-SRV

3.1. Создание локального пользователя sshuser на HQ-SRV и BR-SRV

На HQ-SRV:

Для добавления юзера используется команда `useradd`. Для соблюдения всех условий задания выполним команду `useradd -m -u 2026 -s /bin/bash sshuser`

```
[root@hq-srv ifaces]# useradd -m -u 2026 -s /bin/bash sshuser
[root@hq-srv ifaces]#
```

В этой команде:

- Флаг -m – создает домашнюю директорию для пользователя
- Флаг -u 2026 – задает конкретный UID для пользователя (как требуется по заданию)
- Флаг -s /bin/bash – устанавливает оболочку bash по умолчанию для пользователя
- sshuser – имя пользователя

Теперь следует задать пароль созданному пользователю. Для этого выполняем команду `echo "sshuser:P@ssw0rd" | chpasswd`

```
[root@hq-srv ifaces]# echo "sshuser:P@ssw0rd" | chpasswd
[root@hq-srv ifaces]#
```

Сейчас добавим созданного пользователя в группу wheel (группа пользователей sudo в ALT linux), для этого используем команду `usermod -aG wheel sshuser`.

```
[root@hq-srv ~]# usermod -aG wheel sshuser
[root@hq-srv ~]#
```

После этого выполняем команду `visudo` и у нас открывается конфигурация sudoers. Идем в конец файла и раскомментируем строку `WHEEL_USERS ALL=(ALL:ALL) NOPASSWD: ALL`

```
WHEEL_USERS ALL=(ALL:ALL) NOPASSWD: ALL
```

***Конфигурация откроется в текстовом редакторе vi, которым не все умеют пользоваться. Если вам будет удобнее в другом текстовом редакторе, можете использовать конфиг по пути /etc/sudoers (например, nano /etc/sudoers)**

Можно проверить работу пользователя, для этого вводим команду `su - sshuser`, с помощью нее мы авторизуемся в созданного пользователя, и выполнить команду `sudo`

```
[root@hq-srv ~]# su - sshuser
[sshuser@hq-srv ~]$ sudo
usage: sudo -h | -K | -k | -U
usage: sudo -v [-ABkNnS] [-g group] [-h host] [-p prompt] [-u user]
usage: sudo -l [-ABkNnS] [-g group] [-h host] [-p prompt] [-U user]
        [-u user] [command [arg ...]]
usage: sudo [-ABbEHknPS] [-r role] [-t type] [-C num] [-D directory]
        [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
        [-u user] [VAR=value] [-i | -s] [command [arg ...]]
usage: sudo -e [-ABkNnS] [-r role] [-t type] [-C num] [-D directory]
        [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
        [-u user] file ...
```

Если вам не пишет Permission denied и не запрашивает пароль, то все отлично.

***Те же самые действия выполняем на BR-SRV.**

3.2. Создание net_admin на маршрутизаторах HQ-RTR и BR-RTR

На HQ-RTR:

Вводим команду `useradd -m -s /bin/bash net_admin`

```
root@hq-rtr:/home/user# useradd -m -s /bin/bash net_admin
```

Ставим пароль P@ssw0rd командой `echo "net_admin:P@ssw0rd" | chpasswd`

```
root@hq-rtr:/home/user# echo "net_admin:P@ssw0rd" | chpasswd
```

Дальше нам требуется дать права выполнять команды `sudo` без пароля. Для этого вводим `visudo` и в конец добавляем строку `net_admin ALL=(ALL:ALL) NOPASSWD: ALL`, сохраняемся и выходим из редактора

```
net_admin      ALL=(ALL:ALL) NOPASSWD: ALL
```

На BR-RTR производим те же самые действия.

4. Настройте коммутацию в сегменте HQ

Мы это задание выполнили вместе с настройкой сетевой конфигурации(?).

5. Настройте безопасный удаленный доступ на серверах HQ-SRV и BR-SRV

Для настройки безопасного удаленного доступа будем использовать `ssh`.

*Для настройки `ssh` нам потребуется пакет `openssh-server`, который, как правило, должен быть предустановлен. Если у вас его нет, вы не сможете его скачать, т.к. мы еще не настроили NAT. В таком случае, мой вам совет – отложите это задание до момента, пока у нас не будет доступа в интернет на всех виртуалках.

Настройка на HQ-SRV:

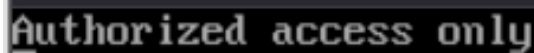
Откроем конфиг по пути `/etc/openssh/sshd_config` и допишем следующие строки:

```
Port 2026
MaxAuthTries 2
PermitRootLogin no
banner /etc/banner
AllowUsers sshuser
```

- Port 2026 – указываем порт, на котором будет прослушиваться `ssh`. Важно при дальнейших подключениях к серверу указывать именно этот порт;
- MaxAuthTries 2 – указываем, сколько раз можно будет ввести пароль пользователю до тех пор, пока его не отключит;
- PermitRootLogin no – отключаем вход для пользователя `root`;

- Banner /etc/banner – указываем путь к баннеру, который будет высвечиваться при попытке подключиться;
- AllowUsers sshuser – указываем, какое имя пользователю нужно использоваться при подключении.

Теперь нам нужно создать баннер по пути /etc/banner, который мы указали в конфиге. Создаем и открываем файл /etc/banner и заполняем его следующим образом

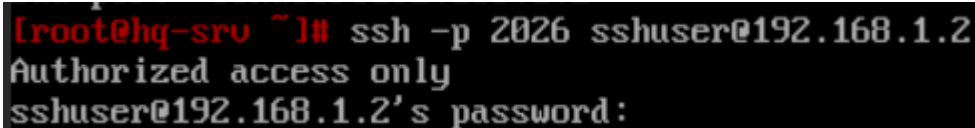


```
Authorized access only
```

Сохраняем и выходим.

Перезагружаем службу ssh командой `systemctl restart sshd`, а также сразу включим автозагрузку командой `systemctl enable --now sshd`.

Для проверки вводим в консоль команду `ssh -p 2026 sshuser@192.168.1.2`



```
[root@hq-srv ~]# ssh -p 2026 sshuser@192.168.1.2
Authorized access only
sshuser@192.168.1.2's password:
```

Можем увидеть, что подключение происходит по правильному порту и появляется баннер, который мы делали ранее.

Теперь повторяем все то же самое на BR-SRV.

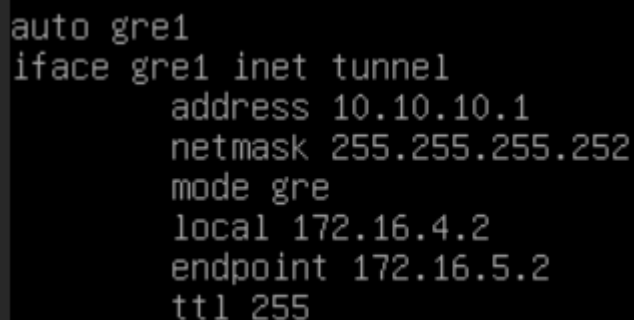
6. Между офисами HQ и BR, на маршрутизаторах HQ-RTR и BR-RTR

необходимо сконфигурировать ip туннель

Туннель будем настраивать при помощи технологии GRE, поскольку он достаточно надежный и удобный.

На HQ-RTR:

Открываем конфиг `/etc/network/interfaces`, в конец файла добавляем строки:



```
auto gre1
iface gre1 inet tunnel
    address 10.10.10.1
    netmask 255.255.255.252
    mode gre
    local 172.16.4.2
    endpoint 172.16.5.2
    ttl 255
```

- Address 10.10.10.1/30 – указываем айпи адрес туннеля на стороне HQ;

- Netmask 255.255.255.252 – указываем 30 маску подсети, поскольку она имеет всего 2 свободных адреса, а больше нам и не надо;
- Mode gre – указываем тип туннеля;
- Local 172.16.4.2 – реальный локальный айпи адрес нашего сервера;
- Endpoint 172.16.5.2 – реальный локальный айпи адрес удаленного сервера;
- Ttl 255 – время жизни пакета. **Обязательно указывайте этот параметр, иначе у вас не будет работать ospf, не повторяйте моих ошибок и не забивайте на него.**

Перезагружаем сеть командой `systemctl restart networking` и проверяем, поднялся ли наш gre интерфейс при помощи команды `ip a`

```
13: gre1@NONE: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1476 qdisc noqueue state UNKNOWN group default qlen 1000
    link/gre 172.16.4.2 peer 172.16.5.2
    inet 10.10.10.1/30 scope global gre1
        valid_lft forever preferred_lft forever
```

Все в порядке, значит приступаем к настройке на втором роутере.

На BR-RTR настройка почти никак не отличается, меняем местами local и endpoint адреса, а также сам айпи адрес туннеля.

```
auto gre1
iface gre1 inet tunnel
    address 10.10.10.2
    netmask 255.255.255.252
    mode gre
    local 172.16.5.2
    endpoint 172.16.4.2
    ttl 255
```

Также перезагружаем сеть, смотрим, что все настройки применились и попробуем выполнить пинг по туннелю с одного роутера на другой

```
root@hq-rtr:/home/user# ping 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=0.731 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.420 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.372 ms
```

Видим, что пинг проходит, а значит настройка туннеля успешно завершена.

7. Обеспечьте динамическую маршрутизацию на маршрутизаторах HQ-RTR и BR-RTR

На обоих роутерах:

Настройку мы будем производить при помощи пакетов frr, которые отсутствуют в репозиториях Astra linux. Поэтому перед началом нам потребуется подключить пакеты великого дебиана. Для этого откроем файл по пути `/etc/apt/sources.list` и добавим туда строку

```
deb [trusted=yes] https://archive.debian.org/debian buster main
```

После этого выполним `apt-get update`, и, вероятно, появится ошибка проверки подписей, но это нам никак не мешает. Теперь выполняем команду `apt-get install frr -y` и дожидаемся конца установки.

После установки, открываем файл по пути `/etc/frr/daemons`, находим строку с `ospfd=no` и ставим `yes`.

```
ospfd=yes
```

Сохраняемся, выходим и выполняем команды `systemctl restart frr` и `systemctl daemon-reload`

```
root@hq-rtr:/home/user# systemctl restart frr
root@hq-rtr:/home/user# systemctl daemon-reload
```

На HQ-RTR:

Пишем в консоль команду `vtysh` и попадаем в интерфейс виртуального роутера

```
root@hq-rtr:/home/user# vtysh
Hello, this is FRRouting (version 6.0.2).
Copyright 1996-2005 Kunihiro Ishiguro, et al.
hq-rtr.au-team.irpo#
```

Теперь нужно вспомнить так всеми любимый `cisco packet tracer`, а точнее команды из него.

Выполняем следующие команды:

Conf t //вход в режим конфигурации роутера

Router ospf // инициализация ospf

Network 10.10.10.0/30 area 0 // указание известных подсетей

Network 192.168.1.0/27 area 0 // указание известных подсетей

Network 192.168.2.0/28 area 0 // указание известных подсетей

Exit //выход из конфигурации ospf

Int gre1 //вход в конфигурацию интерфейса gre1

Ip ospf authentication message-digest //включаем аутентификацию по ключу

*Ip ospf message-digest-key 1 md5 *key //указываем ключ*

End // выход

Write //записываем изменения в память

```
hq-rtr.au-team.irpo# conf t
hq-rtr.au-team.irpo(config)# router ospf
hq-rtr.au-team.irpo(config-router)# network 10.10.10.0/30 area 0
hq-rtr.au-team.irpo(config-router)# network 192.168.1.0/27 area 0
hq-rtr.au-team.irpo(config-router)# network 192.168.2.0/28 area 0
hq-rtr.au-team.irpo(config-router)# exit
hq-rtr.au-team.irpo(config)# int gre1
hq-rtr.au-team.irpo(config-if)# ip ospf authentication message-digest
hq-rtr.au-team.irpo(config-if)# ip ospf message-digest-key 1 md5 arugva
hq-rtr.au-team.irpo(config-if)# end
hq-rtr.au-team.irpo# write
```

На BR-RTR:

Conf t

Router ospf

Network 10.10.10.0/30 area 0

Network 192.168.4.0/28 area 0

Exit

Int gre1

Ip ospf authentication message-digest

*Ip ospf message-digest-key 1 md5 *key*

End

write

```
br-rtr# conf t
br-rtr(config)# router ospf
br-rtr(config-router)# network 10.10.10.0/30 area 0
br-rtr(config-router)# network 192.168.4.0/28 area 0
br-rtr(config-router)# exit
br-rtr(config)# int gre1
br-rtr(config-if)# ip ospf authentication message-digest
br-rtr(config-if)# ip ospf message-digest-key 1 md5 arugva
br-rtr(config-if)# end
br-rtr# write
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
```

*key вписывайте любой, какой вам захочется, главное, чтоб он был одинаковый на обоих роутерах

Теперь можно проверить, определяют ли роутеры друг друга как соседей. Для этого выполним команду `show ip ospf neighbor` на любом из роутеров

```
hq-rtr.au-team.irpo# show ip ospf neighbor
```

Neighbor ID	Pri	State	Dead Time	Address	Interface	RXmtL	RqstL	DBsmL
192.168.4.1	1	Full/DROther	33.717s	10.10.10.2	gre1:10.10.10.1	0	0	0

Видим, что роутер отобразился, а значит все нормально.

8. Настройка динамической трансляции адресов маршрутизаторах HQ-RTR и BR-RTR.

На обоих роутерах:

Для начала разрешим пересылку пакетов ядром, для этого откроем конфиг `/etc/sysctl.conf` и впишем туда строку `net.ipv4.ip_forward=1`, сохраняемся и выходим из конфига, после чего выполняем команду `sysctl -p` для применения правил.

```
root@br-rtr:/home/user# sysctl -p
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv4.ip_forward = 1
```

На HQ-RTR:

Выполняем следующие команды:

```
Iptables -t nat -A POSTROUTING -o *eth0 -s 192.168.1.0/27 -j MASQUERADE
```

```
Iptables -t nat -A POSTROUTING -o *eth0 -s 192.168.2.0/28 -j MASQUERADE
```

```
root@hq-rtr:/home/user# iptables -t nat -A POSTROUTING -o eth0 -s 192.168.1.0/27 -j MASQUERADE
root@hq-rtr:/home/user# iptables -t nat -A POSTROUTING -o eth0 -s 192.168.2.0/28 -j MASQUERADE
root@hq-rtr:/home/user#
```

*eth0 – внешний интерфейс (который смотрит в сторону ISP) вашего роутера

Здесь:

- -t nat – таблица nat (трансляция портов);
- -A POSTROUTING – добавить правило в цепочку POSTROUTING (после маршрутизации);
- -o eth0 – интерфейс, через который будет выходить трафик;
- -s 192.168.1.0/27 – источник трафика;
- -j MASQUERADE – маскировка (подмена исходного айпи на внешний).

Сохраним эти правила при помощи команды `iptables-save`. Перед этим нам нужно создать файл, в который мы будем сохранять эти правила, сделать это можно командой `touch /etc/iptables.rules`. После создания пустого файла, напомним команду `iptables-save > /etc/iptables.rules`.

После каждой перезагрузки виртуалки, где мы создавали правила `iptables` и сохраняли их в файл, нужно будет восстанавливать их командой `iptables-restore < /etc/iptables.rules`, либо же добавить эту команду в `crontab`, чем я заниматься не буду.

Проверим, появился ли интернет на HQ-SRV. Для этого напомним команду `ping 8.8.8.8`

```
[root@hq-srv ~]# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=104 time=48.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=104 time=46.0 ms
```

Интернет есть, все работает отлично.

У вас может быть проблема, что вроде все правильно сделали, но интернета все равно нет. Скорее всего, проблема будет в дефолтном маршруте, а точнее, его отсутствии. Для начала проверьте это командой `ip r`, там должно быть что-то вроде `default via 192.168.1.1`, если этой строки нет, добавьте ее командой `ip r add default via 192.168.1.1`, после этого, если вы все сделали правильно, у вас появится интернет и связь с остальными виртуалками.

На BR-RTR:

Добавим правило `iptables`:

```
Iptables -t nat -A POSTROUTING -o *eth1 -s 192.168.4.0/28 -j MASQUERADE
```

```
root@br-rtr:/home/user# iptables -t nat -A POSTROUTING -o eth1 -s 192.168.4.0/28 -j MASQUERADE
```

После этого проверим, появился ли интернет на BR-SRV

```
(root@br-srv ~)# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=104 time=48.9 ms
```

Также, проверим, правильно ли работает наша динамическая маршрутизация. Попробуем с BR-SRV выполнить icmp запрос к HQ-SRV

```
(root@br-srv ~)# ping 192.168.1.2
PING 192.168.1.2 (192.168.1.2) 56(84) bytes of data.
64 bytes from 192.168.1.2: icmp_seq=1 ttl=62 time=6.46 ms
64 bytes from 192.168.1.2: icmp_seq=2 ttl=62 time=5.28 ms
```

Пинг проходит, все замечательно.

9. Настройте протокол динамической конфигурации хостов для сети в сторону HQ-CLI (иными словами, DHCP)

Настраивать DHCP будем на HQ-RTR, используя isc-dhcp-server, который, на удивление, есть в пакетах астры.

Для начала нам потребуется установить его командой `apt-get install isc-dhcp-server`. После этого, открываем его конфигурацию по пути `/etc/dhcp/dhcpd.conf` и в любом свободном месте дописываем строки:

```
Subnet 192.168.2.0 netmask 255.255.255.240 {
    Range 192.168.2.5 192.168.2.15;
    Option routers 192.168.2.1;
    Option domain-name "au-team.irpo";
    Option domain-name-servers 192.168.1.2;
}
```

```
subnet 192.168.2.0 netmask 255.255.255.240 {
    range 192.168.2.5 192.168.2.15;
    option routers 192.168.2.1;
    option domain-name "au-team.irpo";
    option domain-name-servers 192.168.1.2;
}
```

Теперь открываем файл по пути `/etc/default/isc-dhcp-server` и в строке `INTERFACESv4=""` внутри кавычек вписываем интерфейс, который идет в сторону HQ-CLI, не забывая при этом про VLAN.


```
INTERFACESv4="eth1.200"  
INTERFACESv6=""
```

Осталось только перезагрузить юнит `isc-dhcp-server` и посмотреть статус

```
root@hq-rtr:/home/user# systemctl restart isc-dhcp-server  
root@hq-rtr:/home/user# systemctl status isc-dhcp-server  
● isc-dhcp-server.service - LSB: DHCP server  
   Loaded: loaded (/etc/init.d/isc-dhcp-server; generated; vendor preset: enabled)  
   Active: active (running) since Sun 2025-11-02 20:10:46 +05; 1s ago  
     Docs: man:systemd-sysv-generator(8)  
  Process: 4194 ExecStop=/etc/init.d/isc-dhcp-server stop (code=exited, status=0/SUCCESS)  
  Process: 4210 ExecStart=/etc/init.d/isc-dhcp-server start (code=exited, status=0/SUCCESS)  
    Tasks: 2 (limit: 4915)  
   CGroup: /system.slice/isc-dhcp-server.service  
           └─4225 /usr/sbin/dhcpd -4 -q -cf /etc/dhcp/dhcpd.conf eth1.200  
  
ноя 02 20:10:44 hq-rtr.au-team.irpo systemd[1]: Starting LSB: DHCP server...  
ноя 02 20:10:44 hq-rtr.au-team.irpo isc-dhcp-server[4210]: Launching IPv4 server only.  
ноя 02 20:10:44 hq-rtr.au-team.irpo dhcpd[4224]: Wrote 0 leases to leases file.  
ноя 02 20:10:44 hq-rtr.au-team.irpo dhcpd[4225]: Server starting service.  
ноя 02 20:10:46 hq-rtr.au-team.irpo isc-dhcp-server[4210]: Starting ISC DHCPv4 server: dhcpd.  
ноя 02 20:10:46 hq-rtr.au-team.irpo systemd[1]: Started LSB: DHCP server.  
root@hq-rtr:/home/user#
```

Все отлично, так что идем проверять на HQ-CLI.

Первым делом, перезагружаем сеть командой `systemctl restart network`

```
hq-cli ~ # systemctl restart network
```

сразу после этого смотрим `ip a`

```
hq-cli ~ # ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
   inet 127.0.0.1/8 scope host lo  
       valid_lft forever preferred_lft forever  
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
   link/ether bc:24:11:db:6c:d6 brd ff:ff:ff:ff:ff:ff  
   altnam enp0s18  
   inet6 fe80::dc7a:f7b9:41c7:976c/64 scope link noprefixroute  
       valid_lft forever preferred_lft forever  
4: ens18.200@ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000  
   link/ether bc:24:11:db:6c:d6 brd ff:ff:ff:ff:ff:ff  
   inet 192.168.2.5/28 brd 192.168.2.15 scope global dynamic noprefixroute ens18.200  
       valid_lft 595sec preferred_lft 520sec
```

Видим, что айпи адрес HQ-CLI у нас получил.

На всякий случай также проверим, добавился ли дефолтный маршрут через HQ-RTR. Для этого выполним команду `ip r`

```
hq-cli ~ # ip r  
default via 192.168.2.1 dev ens18.200 proto dhcp src 192.168.2.5 metric 1
```

В моем случае маршрут появился, если у вас его не будет, пропишите команду `ip r add default via 192.168.2.1` и у вас появится интернет и связь с остальной сетью.

Настройка динамической конфигурации хостов окончена.

10. Настройте инфраструктуру разрешения доменных имён для офисов HQ и BR (иными словами DNS)

Основной DNS-сервер у нас располагается на HQ-SRV. Для настройки DNS будем использовать пакет bind.

Для начала, нам требуется установить пакеты bind, которых нет в официальных репозиториях альта, НО, есть в репозиториях яндекса, которые как бы есть в альте, но закомментированы. Что бы их раскомментировать, открываем файл по пути `/etc/apt/sources.list.d/yandex.list` и уберем «#» у трех ссылок на репозитории, который начинаются с `http://` (по моим наблюдениям они работают лучше, чем `ftp`)

```
rpm [p10] http://mirror.yandex.ru/altlinux p10/branch/x86_64 classic gostcrypto
rpm [p10] http://mirror.yandex.ru/altlinux p10/branch/x86_64-i586 classic
rpm [p10] http://mirror.yandex.ru/altlinux p10/branch/noarch classic
```

После этого выполняем `apt-get update` и ждем обновления репозиториев.

Теперь можем спокойно установить пакеты bind командой `apt-get install bind`.

Наконец, можем начинать настройку. Первым делом, нужно объявить зоны, которые у нас будут задействованы. Для этого открываем конфигурационный файл по пути `/etc/bind/local.conf` и заполняем по следующему примеру:

```
zone "au-team.irpo" {
    type master;
    file "/etc/bind/db.au-team.irpo";
};

zone "1.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/db.1.168.192.in-addr.arpa";
};

zone "2.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/db.2.168.192.in-addr.arpa";
};
```

Здесь:

- zone “au-team.irpo” – объявление прямой зоны ДНС;
- type master – Указываем, что это основной сервер для данной зоны;
- file “...” – указываем путь к конфигурации зоны;
- zone “1.168.192.in-addr.arpa” и zone “2.168.192.in-addr.arpa” – объявление обратных зон.

Теперь создаем конфигурации самих зон.

Начнем с прямой зоны. Скопируем образец файла зоны `/bind/zone/empty` командой `cp /etc/bind/zone/empty /etc/bind/db.au-team.irpo`

```
[root@hq-srv ~]# cp /etc/bind/zone/empty /etc/bind/db.au-team.irpo
```

Теперь откроем конфиг любым удобным нам редактором и заполним его по следующему примеру:

```
; Instead, copy it and use that copy.
;
$TTL      1D
@         IN      SOA      hq-srv.au-team.irpo. admin.au-team.irpo. (
                                2025020600      ; serial
                                12H              ; refresh
                                1H              ; retry
                                1W              ; expire
                                1H              ; ncache
        )
        IN      NS       hq-srv.au-team.irpo.
hq-srv    IN      A       192.168.1.2
hq-cli    IN      A       192.168.2.5
hq-rtr    IN      A       192.168.1.1
br-rtr    IN      A       192.168.4.1
br-srv    IN      A       192.168.4.2
web       IN      A       172.16.5.1
decker    IN      A       172.16.4.1
```

Здесь:

- SOA hq-srv.au-team.irpo – указываем главный DNS-сервер;
- Запись NS hq-srv.au-team.irpo – сервер имен для домена;
- @ – Указываем главный сервер;
- hq-cli IN A 192.168.2.5 – соотносим имя и IP-адрес устройства;
- И так далее...

С прямой зоной закончили, теперь обратные зоны. Копируем образец конфига командой `cp /etc/bind/zone/127.in-addr.arpa /etc/bind/db.1.168.192.in-addr.arpa` и сразу файл второй обратной зоны `cp/etc/bind/zone/127.in-addr.arpa /etc/bind/db.2.168.192.in-addr.arpa`. Открываем файл обратной зоны для подсети 192.168.1.0 и заполняем следующим образом:

```
$TTL      1D
@         IN      SOA      hq-srv.au-team.irpo. admin.au-team.irpo. (
                                2025020600      ; serial
                                12H              ; refresh
                                1H              ; retry
                                1W              ; expire
                                1H              ; ncache
        )
        IN      NS       hq-srv.au-team.irpo.
1         IN      PTR      hq-rtr.au-team.irpo.
2         IN      PTR      hq-cli.au-team.irpo.
```

Здесь:

- 1 IN PTR hq-rtr.au-team.irpo соотносит адрес 192.168.1.1 с именем
- 2 IN PTR hq-srv.au-team.irpo соотносит адрес 192.168.1.2 с именем

Обратите внимание, что в первой колонке конфигурации пишется последний октет айпи адреса устройства, имя которого мы вносим.

По такой же аналогии заполняем второй файл обратной зоны db.2.168.192.in-addr.arpa

```
$TTL      1D
@         IN      SOA      hq-srv.au-team.irpo. admin.au-team.irpo. (
                                2025020600    ; serial
                                12H             ; refresh
                                1H             ; retry
                                1W             ; expire
                                1H             ; ncache
                                )
         IN      NS       hq-srv.au-team.irpo.
1         IN      PTR      hq-rtr.au-team.irpo
5         IN      PTR      hq-cli.au-team.irpo.
```

Настройка зон окончена. Теперь можем проверить наши конфиги на ошибки при помощи встроенных в bind команд `named-checkconf` и `named-checkzone`.

Сначала выполняем `named-checkconf`

```
[root@hq-srv ~]# named-checkconf
[root@hq-srv ~]#
```

Если он ничего не вывел – отлично, так и должно быть.

Теперь проверяем непосредственно файлы зон. Для этого вводим команды:

- `named-checkzone au-team.irpo /etc/bind/db.au-team.irpo`
- `named-checkzone 1.168.192.in-addr.arpa /etc/bind/db.1.168.192.in-addr.arpa`
- `named-checkzone 2.168.192.in-addr.arpa /etc/bind/db.2.168.192.in-addr.arpa`

```
[root@hq-srv ~]# named-checkzone au-team.irpo /etc/bind/db.au-team.irpo
zone au-team.irpo/IN: loaded serial 2025020600
OK
[root@hq-srv ~]# named-checkzone 1.168.192.in-addr.arpa /etc/bind/db.1.168.192.in-addr.arpa
zone 1.168.192.in-addr.arpa/IN: loaded serial 2025020600
OK
[root@hq-srv ~]# named-checkzone 2.168.192.in-addr.arpa /etc/bind/db.2.168.192.in-addr.arpa
zone 2.168.192.in-addr.arpa/IN: loaded serial 2025020600
OK
[root@hq-srv ~]#
```

В случае с проверками зон везде должно вывести ОК, и, если вы все делали правильно, у вас так и будет. Теперь перезагрузим службу bind командой `systemctl restart bind` и попробуем пингануть, к примеру, самого себя по доменному имени, но, возможно, столкнемся с ошибкой. Далее подробнее об этом.

В величайшем Альт линукс bind работает из-под встроенного пользователя `named`, соответственно, файлы зон должны принадлежать группе `named`, однако же, по непонятной мне причине, они создаются с владельцем `root` – это допустимо, и группой

root – это не допустимо. Так что теперь нам потребуется изменить группу владельцев файлов конфигурации на named, для этого вводим в консоль команду `chown root:named /etc/bind/*`, вновь перезагружаем службу и проверяем командой `dig hq-srv.au-team.irpo`

```
[root@hq-srv ~]# dig hq-srv.au-team.irpo

; <<>> DiG 9.16.50 <<>> hq-srv.au-team.irpo
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17003
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: f440b668e2890c6d010000006907991f7aa5c69e031ec161 (good)
;; QUESTION SECTION:
;hq-srv.au-team.irpo.                IN      A
;; ANSWER SECTION:
hq-srv.au-team.irpo.      86400   IN      A      192.168.1.2
;; Query time: 1 msec
;; SERVER: 192.168.1.2#53(192.168.1.2)
;; WHEN: Sun Nov 02 22:47:11 +05 2025
;; MSG SIZE rcvd: 92
```

Ошибка с правами очень странная и довольно нетипичная, лично я за десятки раз настройки bind на разных дистрибутивах столкнулся с ней впервые.

Теперь нам потребуется на каждом устройстве открыть конфиг `/etc/resolv.conf` и ввести туда две строки:

- `nameserver 192.168.1.2`
- `search au-team.irpo`

```
nameserver 192.168.1.2
search au-team.irpo
```

Важно! Если хотите, чтобы у вас нормально функционировал интернет, то в этот конфиг надо еще дописать `nameserver 8.8.8.8`, главное, чтоб эта запись была ниже остальных. Либо же можно настроить наш сервер на рекурсию, как это сделать описано в пункте 0.2.

11. Настройте часовой пояс на всех устройствах согласно месту проведения экзамена.

Крайне просто задание. Для его выполнения воспользуемся утилитой `timedatectl` с использованием команды `set-timezone`.

На каждой виртуальной машине вводим в консоль следующую команду:

```
timedatectl set-timezone Asia/Yekaterinburg
```

```
root@br-rtr:/home/user# timedatectl set-timezone Asia/Yekaterinburg
```

Проверяем, применились ли настройки, выполнив команду `timedatectl`

```
root@br-rtr:/home/user# timedatectl
      Local time: Bc 2025-11-02 23:01:33 +05
      Universal time: Bc 2025-11-02 18:01:33 UTC
          RTC time: Bc 2025-11-02 18:01:33
      Time zone: Asia/Yekaterinburg (+05, +0500)
Network time on: yes
NTP synchronized: no
  RTC in local TZ: no
root@br-rtr:/home/user#
```

Поздравляю, задание выполнено и первый модуль подошел к концу.

МОДУЛЬ 2

1. Настройте контроллер домена Samba DC на сервере BR-SRV

Для выполнения этого задания нам потребуется установить пакет `task-samba-dc`, включающий для себя все необходимые утилиты для разворачивания домена на линукс.

```
[root@br-srv ~]# apt-get install task-samba-dc
Reading Package Lists... Done
Building Dependency Tree... Done
task-samba-dc is already the newest version.
0 upgraded, 0 newly installed, 0 removed and 0 not upgraded.
```

После этого обязательно нужно удалить стандартный конфигурационный файл `smb.conf`, для этого выполним команду `rm -f /etc/samba/smb.conf`

```
[root@br-srv ~]# rm -f /etc/samba/smb.conf
[root@br-srv ~]# ls /etc/samba/smb.conf
ls: cannot access '/etc/samba/smb.conf': No such file or directory
```

Теперь можем начинать инициализацию домена. Для этого выполним команду `samba-tool domain provision`. В процессе создания домена нам потребуется отвечать на некоторые вопросы:

```

ls: cannot access /etc/samba/smb.conf: no such file or directory
[root@br-srv ~]# samba-tool domain provision
Realm [AU-TEAM.IRPO]:
Domain [AU-TEAM]:
Server Role (dc, member, standalone) [dc]:
DNS backend (SAMBA_INTERNAL, BIND9_FLATFILE, BIND9_DL2, NONE) [SAMBA_INTERNAL]:
DNS forwarder IP address (write 'none' to disable forwarding) [192.168.1.2]:
Administrator password:
Retype password:

```

- Realm AU-TEAM.IRPO – указываем полное имя домена заглавными буквами;
- Domain AU-TEAM – указываем короткое (NetBios) имя домена;
- Server Role dc – указываем, что наш сервер выступает в роли контроллера домена (dc – domain controller);
- DNS backend SAMBA_INTERNAL – указываем тип DNS сервера, в нашем случае встроенный днс самбы, он самый удобный;
- DNS forwarder 192.168.1.2 – Перенаправляет пакеты для внешнего доступа на HQ-SRV;
- Также два раза вводим пароль администратора домена.

После недолгого ожидания мы увидим такие строки, они означают, что домен создан успешно

```

INFO 2025-11-03 00:01:34,932 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #2432: A Kerberos configuration suitable for Samba AD has been g
enerated at /var/lib/samba/private/krb5.conf
INFO 2025-11-03 00:01:34,951 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #2434: Merge the contents of this file with your system krb5.con
f or replace it with this one. Do not create a symlink!
INFO 2025-11-03 00:01:35,252 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #493: Once the above files are installed, your Samba AD server w
ill be ready to use
INFO 2025-11-03 00:01:35,270 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #498: Server Role:          active directory domain controller
INFO 2025-11-03 00:01:35,280 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #499: Hostname:          br-srv
INFO 2025-11-03 00:01:35,291 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #500: NetBIOS Domain:    AU-TEAM
INFO 2025-11-03 00:01:35,301 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #501: DNS Domain:      au-team.irpo
INFO 2025-11-03 00:01:35,311 pid:4805 /usr/lib64/samba-dc/python3.9/samba/provision/__init__.py #502: DOMAIN SID:      S-1-5-21-1270499769-4288854768-3799
641975

```

После этого в любом удобном вам текстовом редакторе открываем конфиг `/etc/samba/smb.conf`, которой вновь созданся и заполнился после создания домена и в `[global]` дописываем строку `interfaces = 192.168.4.2, 127.0.0.1`

```

[global]
    dns forwarder = 192.168.1.2
    netbios name = BR-SRV
    realm = AU-TEAM.IRPO
    server role = active directory domain controller
    workgroup = AU-TEAM
    interfaces = 192.168.4.2, 127.0.0.1

```

Перезагружаем samba командой `systemctl restart samba` и заодно сразу включим в автозагрузку командой `systemctl enable --now samba`.

Сейчас можем проверить, нормально ли функционирует домен, для этого выполним команду `samba-tool domain info 127.0.0.1`

```
[root@br-srv ~]# samba-tool domain info 127.0.0.1
Forest                : au-team.irpo
Domain                : au-team.irpo
Netbios domain       : AU-TEAM
DC name               : br-srv.au-team.irpo
DC netbios name      : BR-SRV
Server site           : Default-First-Site-Name
Client site           : Default-First-Site-Name
```

Если вывод такой – значит пока все нормально, двигаемся дальше

Далее копируем конфиг Kerberos, который создавался автоматически при создании домена, командой `cp /var/lib/samba/private/krb5.conf /etc/krb5.conf`

```
[root@br-srv ~]# cp /var/lib/samba/private/krb5.conf /etc/krb5.conf
cp: overwrite '/etc/krb5.conf'? yes
[root@br-srv ~]#
```

Следующим шагом нам требуется ввести адрес контроллера домена в правила пересылки пакетов на DNS-сервере, что бы все пакеты к домену au-team.irpo перенаправлялись на контроллер домена. Для этого, открываем HQ-SRV, в любом текстовом редакторе открываем конфиг `/etc/bind/options.conf`, находим строку `forwarders { 8.8.8.8; }` и перед закрывающей скобкой дописываем адрес BR-SRV – 192.168.4.2

```
forwarders { 8.8.8.8; 192.168.4.2; };
```

***После адреса обязательно должен быть знак «;»**

*Я не уверен на 100%, что этот вариант сработает, поэтому лучше на всякий случай добавить строку `forwarders { 192.168.4.2; };` в конфиг `/etc/bind/local.conf` в блок с нашей прямой зоной

```
zone "au-team.irpo" {
    type master;
    forwarders { 192.168.4.2; };
    file "/etc/bind/db.au-team.irpo";
};
```

Сохраняем конфиг, выходим и перезагружаем службу bind командой `systemctl restart bind`.

Теперь можем попробовать ввести в домен клиентскую машину HQ-CLI. Для этого перейдем на нее, откроем консоль и впишем команду `/usr/sbin/realmd join -U Administrator`

*HQ-CLI сам должен увидеть домен, поэтому в команде мы его явно не указали. Если у вас так не работает, используйте команду `/usr/sbin/realm join -U Administrator AU-TEAM.IRPO`

```
hq-cli ~ # /usr/sbin/realm join -U Administrator
Пароль для Administrator:
```

После того, как мы ввели пароль администратора, мы оказываемся в домене. Проверить это можно, используя команду `/usr/sbin/realm list`

```
hq-cli ~ # /usr/sbin/realm list
au-team.irpo
  type: kerberos
  realm-name: AU-TEAM.IRPO
  domain-name: au-team.irpo
  configured: kerberos-member
  server-software: active-directory
  client-software: sssd
  required-package: sssd-tools
  required-package: sssd
  required-package: adcli
  required-package: samba-common-tools
  login-formats: %U@au-team.irpo
  login-policy: allow-realm-logins
```

После ввода HQ-CLI в домен, нам требуется создать 5 доменных пользователей. Для этого перейдем обратно на BR-SRV и выполним команду `samba-tool user create hquser1-5 P@ssw0rd`

```
[root@br-srv ~]# samba-tool user create hquser1 P@ssw0rd
User 'hquser1' added successfully
[root@br-srv ~]# samba-tool user create hquser2 P@ssw0rd
User 'hquser2' added successfully
[root@br-srv ~]# samba-tool user create hquser3 P@ssw0rd
User 'hquser3' added successfully
[root@br-srv ~]# samba-tool user create hquser4 P@ssw0rd
User 'hquser4' added successfully
[root@br-srv ~]# samba-tool user create hquser5 P@ssw0rd
User 'hquser5' added successfully
```

Можем проверить, точно ли пользователи создались, выполнив команду `samba-tool user list`

```
[root@br-srv ~]# samba-tool user list
hquser1
hquser2
Administrator
hquser3
hquser4
Guest
krbtgt
hquser5
```

Пользователи созданы, теперь нужно создать группу hq. Для этого выполняем команду `samba-tool create group add hq`. После успешного создания группы добавляем в нее пользователей. Это можно сделать одной командой `samba-tool group addmembers hq hquser1,hquser2,hquser3,hquser4,hquser5`

```
[root@br-srv ~]# samba-tool group addmembers hq hquser1,hquser2,hquser3,hquser4,hquser5
Added members to group hq
```

Сейчас с HQ-CLI проверим вход в систему из-под созданных пользователей. Перед началом нужно перезагрузить виртуальную машину.

Честно, я до конца так и не понял, как тут работает вход в систему с доменных пользователей, поскольку работает оно через раз. Для начала попробуем просто сразу войти в пользователя, вписав в консоль команду `su - hquser1`. Если же ничего не получилось – открываем конфиг `/etc/sss/sssd.conf` и приведем к следующему виду

```
[sssd]
domains = au-team.irpo
config_file_version = 2
services = nss, pam

[domain/au-team.irpo]
default_shell = /bin/bash
krb5_store_password_if_offline = True
cache_credentials = True
krb5_realm = AU-TEAM.IRPO
realmd_tags = manages-system joined-with-adcli
id_provider = ad
fallback_homedir = /home/%u
ad_domain = au-team.irpo
use_fully_qualified_names = False
ldap_id_mapping = True
access_provider = ad
override_homedir = /home/%u
auth_provider = ad
dyndns_update = false
```

Самое главное, что тут требуется изменить, это:

- `Ldap_id_mapping = true;`
- `Use_fully_qualified_names = false;`

- `Fallback_homedir = /home/%u` (нужно убрать `@`, поскольку в нашем «самом лучшем» дистрибутиве не поддерживается, и, по этой причине, `sssd` не может создать каталог).

Остальные параметры у вас должны изначально соответствовать моим, если это не так, исправьте.

После этого перезагружаем службу командой `systemctl restart sssd` и пробуем войти в доменную учетную запись пользователя командой `su - hquser1`

```
hq-cli ~ # su - hquser1
hquser1@hq-cli ~ $
```

Хвала богам, у меня это получилось. Однако, никто не может гарантировать, что это получится у вас, т.к. в альт линукс очень много различных проблем, которые могут вам очень сильно помешать.

Теперь нам потребуется настроить группу `hq` так, чтобы они могли использовать с `sudo` исключительно команды `cat`, `grep` и `id`. Задание достаточно-таки расплывчатое, не понятно, где и как его выполнять, а потому, мы выполним его на HQ-CLI, поскольку это будет легче, чем мучаться на самом контроллере.

На HQ-CLI создаем группу `hq`, для этого вводим команду `/usr/sbin/groupadd hq`

```
hq-cli ~ # /usr/sbin/groupadd hq
groupadd: группа «hq» уже существует
hq-cli ~ #
```

Потом добавляем в нее пользователей, для этого используем команду `gpasswd -a 'hquser№' hq`

```
hq-cli ~ # gpasswd -a 'hquser1' hq
Добавление пользователя hquser1 в группу hq
hq-cli ~ # gpasswd -a 'hquser2' hq
Добавление пользователя hquser2 в группу hq
hq-cli ~ # gpasswd -a 'hquser3' hq
Добавление пользователя hquser3 в группу hq
hq-cli ~ # gpasswd -a 'hquser4' hq
Добавление пользователя hquser4 в группу hq
hq-cli ~ # gpasswd -a 'hquser5' hq
Добавление пользователя hquser5 в группу hq
```

Проверяем состав группы командой `getent group hq`

```
groupadd: группа «hq» уже существует
hq-cli ~ # getent group hq
hq:x:501:hquser1,hquser2,hquser3,hquser4,hquser5
hq-cli ~ #
```

Пользователи в группу добавлены, значит можем приступать к следующему шагу.

Вводим `/usr/sbin/visudo`, у нас откроется конфигурационный файл `sudoers`, идем в конец и вводим строку

```
%hq ALL=(ALL) NOPASSWD: /usr/bin/cat, /bin/grep, /usr/bin/id
```

Также, как оказалось, в этом самом лучшем моем любимом дистрибутиве, из коробки имеются проблемы с правами на бинарник `sudo` и файл `/etc/sudoers`, из-за чего, нашим доменным пользователям вообще нельзя будет использовать `sudo`. Что бы исправить эти проблемы, из-под рута выполняем команду `chmod 4755 /usr/bin/sudo`

```
hq-cli ~ # chmod 4755 /usr/bin/sudo
hq-cli ~ # ls -l /usr/bin/sudo
-rwsr-xr-x 1 root wheel 186928 сен 12 2021 /usr/bin/sudo
```

И команду `chmod 440 /etc/sudoers`

```
hq-cli ~ # chmod 440 /etc/sudoers
hq-cli ~ # ls -l /etc/sudoers
-r--r----- 1 root root 3612 ноя 3 05:51 /etc/sudoers
```

И вот теперь, наконец то, мы можем авторизоваться под любого из `hquser` и проверить результат нашей работы. Для этого в консоли выполняем команду `su - hquser1` и используем команду `sudo cat` на любой файл (например, `cat /etc/sysctl.conf`)

```
hquser1@hq-cli ~ $ sudo cat /etc/sysctl.conf
# sysctl settings are defined through files in
# /lib/sysctl.d/, /run/sysctl.d/, and /etc/sysctl.d/.
#
# Vendors settings live in /lib/sysctl.d/.
# To override a whole file, create a new file with the same in
# /etc/sysctl.d/ and put new settings there. To override
# only specific settings, add a file with a lexically later
# name in /etc/sysctl.d/ and put new settings there.
#
# For more information, see sysctl.conf(5) and sysctl.d(5).
net.ipv6.conf.all.disable_ipv6=1
net.ipv6.conf.default.disable_ipv6=1
```

Команда работает, остальные команды (`grep`, `id`) можете проверить сами. Теперь пробуем выполнить любую запрещенную команду, например, `sudo ls`


```
hqluser1@hq-cli ~ $ sudo ls /etc/
[sudo] password for hqluser1:
Sorry, user hqluser1 is not allowed to execute '/bin/ls /etc/' as root on hq-cli.au-team.irpo.
```

Видим, что команда не работает, поскольку она запрещена.

*Этот пункт задания выполнен совсем через костыль, и я не уверен, что на демоэкзамене зачтут такое выполнения, хотя, по факту, все условия задания соблюдены, да и вряд ли кто-то будет это проверять.

*Мой вам совет, на самом демоэкзамене пропускайте это задание, мало того, что оно в целом занимает много времени, так у вас еще может все получиться далеко не с первого раза, т.к. пока неизвестно какие версии дистрибутивов у нас будут использоваться, а на каждой версии свои проблемы, над решением которых я порой сидел не один десяток часов (например, на последних версиях дистрибутивов альт сервер и альт воркстейшен я так и не смог поднять и нормально настроить samba dc).

На этом первое задание второго модуля окончено, можем двигаться дальше.

2. Сконфигурируйте файловое хранилище на сервере HQ-SRV

В данном задании мы настроим программный RAID уровня 0. Для этого в proxmox перейдем в hardware на HQ-SRV и добавим 2 диска, каждый объемом 1ГБ.

	Hard Disk (scsi0)	local-lvm:vm-103-disk-0,iouthread=1,size=32G
	Hard Disk (scsi1)	local-lvm:vm-103-disk-1,iouthread=1,size=1G
	Hard Disk (scsi2)	local-lvm:vm-103-disk-2,iouthread=1,size=1G

Теперь определим их названия в самой системе, для этого в консоли выполним команду `lsblk`

```
[root@hq-srv ~]# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   32G  0 disk
├─sda1       8:1    0   490M  0 part [SWAP]
└─sda2       8:2    0  31.5G  0 part /
sdb          8:16    0    1G  0 disk
sdc          8:32    0    1G  0 disk
sr0         11:0    1   4.8G  0 rom
```

Можем увидеть, что диски называются sdb и sdc (в вашем случае могут отличаться, смотрите внимательно).

Далее выполняем команду `mdadm --create /dev/md0 --level=0 --raid-device=2 /dev/sdb /dev/sdc`

```
[root@hq-srv ~]# mdadm --create /dev/md0 --level=0 --raid-device=2 /dev/sdb /dev/sdc
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

Где:

- /dev/md0 – путь и название рейд массива;
- --level=0 – Уровень рейда, который мы будем использовать;
- --raid-device = 2 – кол-во дисков, который будут объединены в массив;
- --dev/sdb /dev/sdc – диски, которые мы собираемся объединить.

Теперь нам нужно создать раздел на нашем рейд массиве. Для этого выполним команду `fdisk /dev/md0`. У нас запустится утилита, в которую по мере продвижения нам нужно будет отправлять команды

```
Welcome to fdisk (util-linux 2.39.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x267697eb.

Command (m for help): _
```

После запуска утилиты отправляем следующие команды:

- Command > N – new partition – создаем новый раздел;
- Partition type > P – Primary – выбираем тип Primary для раздела;
- Потом 3 раза жмем энтер, там нам ничего менять не нужно.

```
Command (m for help): n
Partition type
  p   primary (0 primary, 0 extended, 4 free)
  e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1):
First sector (2048-4186111, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-4186111, default 4186111):

Created a new partition 1 of type 'Linux' and of size 2 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

После этого, отправляем команду `w`, которая записывает таблицу разделов на диск, применяет все изменения и завершает работу `fdisk`.

Сейчас нам нужно переформатировать раздел в файловую систему ext4. Для выполняем команду `mkfs.ext4 /dev/md0p1`

```
[root@hq-srv ~]# mkfs.ext4 /dev/md0p1
mke2fs 1.46.2 (28-Feb-2021)
Discarding device blocks: done
Creating filesystem with 523008 4k blocks and 130816 inodes
Filesystem UUID: dfdbc864-275e-4305-8994-359d86f06f90
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
```

Последний шагом нужно куда-то монтировать наш массив. Для этого создадим директорию `/raid/` командой `mkdir /raid/` и введем команду `mount /dev/md0p1 /raid`

```
hq-srv ~]# mkdir /raid/
hq-srv ~]# mount /dev/md0p1 /raid
```

Проверяем командой `lsblk`

```
[root@hq-srv ~]# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
sda          8:0    0   32G  0 disk
├─sda1       8:1    0   490M  0 part  [SWAP]
└─sda2       8:2    0  31.5G  0 part  /
sdb          8:16   0    1G   0 disk
├─md0        9:0    0    2G   0 raid0
└─md0p1      259:0   0    2G   0 part  /raid
sdc          8:32   0    1G   0 disk
├─md0        9:0    0    2G   0 raid0
└─md0p1      259:0   0    2G   0 part  /raid
sr0         11:0    1   4.8G  0 rom
```

3. Настройте сервер сетевой файловой системы (nfs) на HQ-SRV.

Первым делом на требуется установить пакет `nfs-server`, для этого в консоль вводим `apt-get install nfs-server`. После установки создаем директорию `/raid/nfs`, которая у нас будет использоваться как общая и сразу выдаем ей права `777` командой `chmod 777 /raid/nfs`


```
[root@hq-srv ~]# mkdir -p /raid/nfs
[root@hq-srv ~]# chmod 777 /raid/nfs
```

После этого откроем конфигурационный файл `/etc/exports` и впишем туда строку `/raid/nfs 192.168.2.0/28(rw,no_subtree_check)`

```
/raid/nfs 192.168.2.0/28(rw,no_subtree_check)
```

Где:

- `/raid/nfs` – директория, которую расшариваем;
- `192.168.2.0/28` – подсеть, которой разрешен доступ;
- `Rw` – права доступа на чтение и запись;
- `No_subtree_check` – отключаем проверку поддиректорий для повышения производительности.

Теперь применяем конфигурацию, введя команду `exportfs -a`, а также сразу проверим командой `exportfs -v`

```
[root@hq-srv ~]# exportfs -a
[root@hq-srv ~]# exportfs -v
/raid/nfs 192.168.2.0/28(sync,wdelay,hide,no_subtree_check,sec=sys,rw,root_squash,no_all_squash)
/srv/public <world>(sync,wdelay,hide,no_subtree_check,fsid=1,sec=sys,ro,root_squash,no_all_squash)
```

После этого перезагружаем юнит `nfs-server`, а также добавляем его в автозагрузку

```
[root@hq-srv ~]# systemctl restart nfs-server
[root@hq-srv ~]# systemctl enable --now nfs-server
```

***ВАЖНО!** Если вы настраивали `nat` так, как это делал я, то скорее всего вам придется добавить еще одно правило `iptables` на `HQ-RTR`, т.к. он по какой-то причине маскардит пакеты с источником `ens18.200` и назначением `ens18.100`. Что бы это исправить, добавьте на `HQ-RTR` следующее правило: `iptables -t nat -I POSTROUTING -s 192.168.2.0/28 -d 192.168.1.0/27 -j ACCEPT`. Скорее всего, это моя ошибка, позже исправлю настройку `NAT` на более правильную.

Теперь перейдем к настройке с клиента, которым у нас выступает `HQ-CLI`.
Инсталлируем на него пакет `nfs-clients` и создадим директорию `/mnt/nfs/` командой

`mkdir -p /mnt/nfs/`. После этого, пытаемся вручную монтировать nfs, для этого вводим команду `mount -t nfs HQ-SRV:/raid/nfs /mnt/nfs`

```
hq-cli ~ # mount -t nfs HQ-SRV:/raid/nfs /mnt/nfs -v
mount.nfs: timeout set for Mon Nov  3 11:04:20 2025
mount.nfs: trying text-based options 'vers=4.2,addr=192.168.1.2,clientaddr=192.168.2.5'
```

*у вас не должно быть никакого вывода, если вы не ставили флаг -v

Также, выполним команду `df -h`, чтобы убедиться, что nfs монтировался как надо и куда надо

```
hq-cli ~ # df -h
Файловая система  Размер  Использовано  Дост  Использовано%  Смонтировано в
udevfs             5,0M      96K          5,0M           2% /dev
runfs              991M     1016K        990M           1% /run
/dev/sda2          30G       7,2G         21G          26% /
tmpfs              991M           0        991M           0% /dev/shm
tmpfs              991M      8,0K        991M           1% /tmp
tmpfs              199M      84K         198M           1% /run/user/500
tmpfs              199M      56K         198M           1% /run/user/0
HQ-SRV:/raid/nfs   2,0G           0         1,9G           0% /mnt/nfs
```

Можем увидеть, что монтирование произошло успешно. Теперь настроим автомонтирование. Делать это мы будем при помощи конфигурационного файла `/etc/fstab`, откроем его в любом удобном текстовом редакторе и в конец допишем строчку `HQ-SRV:/raid/nfs /mnt/nfs nfs defaults 0 0`

```
HQ-SRV:/raid/nfs      /mnt/nfs      nfs      defaults      0      0
```

Обратите внимание, что для отступов используется Tab, а не Space.

После добавления этой строки первым делом размонтируем `/mnt/nfs` командой `umount /mnt/nfs`, сразу после чего введем команду `mount -a`

```
hq-cli ~ # nano /etc/fstab
hq-cli ~ # umount /mnt/nfs
hq-cli ~ # mount -a
```

Теперь проверим, произошло ли автоматическое монтирование, для этого введем `df -h`

```

hq-cli ~ # df -h

```

Файловая система	Размер	Использовано	Дост	Использовано%	Смонтировано в
udevfs	5,0M	96K	5,0M	2%	/dev
runfs	991M	1016K	990M	1%	/run
/dev/sda2	30G	7,2G	21G	26%	/
tmpfs	991M	0	991M	0%	/dev/shm
tmpfs	991M	8,0K	991M	1%	/tmp
tmpfs	199M	84K	198M	1%	/run/user/500
tmpfs	199M	56K	198M	1%	/run/user/0
HQ-SRV:/raid/nfs	2,0G	0	1,9G	0%	/mnt/nfs

Можем увидеть, что все монтировалось как надо. Последним шагом проверим создание файлов в общей папке, для этого на клиенте создадим в директории /mnt/nfs любой файл при помощи команды `touch`

```

hq-cli ~ # touch /mnt/nfs/alt-linux-is-bad
hq-cli ~ # ls /mnt/nfs
alt-linux-is-bad

```

Теперь на сервере в директории /raid/nfs проверим наличие этого файла при помощи команды `ls`

```

[root@hq-srv ~]# ls /raid/nfs/
alt-linux-is-bad

```

4. Настройте службу сетевого времени на базе сервиса chrony на маршрутизаторе ISP

На ISP обновляем пакеты командой `apt-get update`, после чего устанавливаем пакет `chrony` командой `apt-get install chrony`. После установки в любом текстовом редакторе откроем конфиг по пути `/etc/chrony/chrony.conf` и изменим следующим образом:

```
# Welcome to the chrony configuration file. See chrony.conf(5) for more
# information about usable directives.
#pool 2.debian.pool.ntp.org iburst

# This directive specify the location of the file containing ID/key pairs for
# NTP authentication.
keyfile /etc/chrony/chrony.keys

# This directive specify the file into which chronyd will store the rate
# information.
driftfile /var/lib/chrony/chrony.drift

# Uncomment the following line to turn logging on.
#log tracking measurements statistics

# Log files location.
logdir /var/log/chrony

# Stop bad estimates upsetting machine clock.
maxupdateskew 100.0

# This directive enables kernel synchronisation (every 11 minutes) of the
# real-time clock. Note that it can't be used along with the 'rtcfile' directive.
#rtcsync

# Step the system clock instead of slewing it if the adjustment is larger than
# one second, but only in the first three clock updates.
makestep 1 3

local stratum 5
allow 172.16.4.0/28
allow 172.16.5.0/28
allow 192.168.1.0/27
allow 192.168.2.0/28
allow 192.168.4.0/27
```

А точнее:

Закомментируем строки pool и rtcsync, поскольку они будут нам мешать.

Добавим в конце конфига строки:

Local stratum 5 // указывает 5 уровень иерархии (что бы это ни значило:)

Allow 172.16.4.0/28 // разрешает доступ к NTP серверу данной подсети

Allow 172.16.5.0/28

Allow 192.168.1.0/27

Allow 192.168.2.0/28

Allow 192.168.4.0/27

Перезагружаем службу командой `systemctl restart chronyd`.

Действия, которые требуется выполнить на каждом устройстве:

Прописываем команду `timedatectl set-ntp false`, чтобы ISP у нас был главным сервером времени. После этого открываем конфиг по пути `/etc/chrony.conf`,

комментируем строки `rtcsync` и `pool pool.ntp.org iburst`, а также дописываем строку `server 172.16.4.1 iburst`

```
# Use public servers from the pool.ntp.org project.
# Please consider joining the pool (https://www.pool.ntp.org/join.html).
#pool pool.ntp.org iburst
server 172.16.4.1 iburst
# Record the rate at which the system clock gains/losses time.
driftfile /var/lib/chrony/drift

# Allow the system clock to be stepped in the first three updates
# if its offset is larger than 1 second.
makestep 1.0 3

# Enable kernel synchronization of the real-time clock (RTC).
#rtcsync

# Enable hardware timestamping on all interfaces that support it.
#hwtimestamp *

# Increase the minimum number of selectable sources required to adjust
# the system clock.
#minsources 2

# Allow NTP client access from local network.
#allow 192.168.0.0/16

# Serve time even if not synchronized to a time source.
#local stratum 10
```

После чего перезагружаем командой `systemctl restart chronyd`.

5. Сконфигурируйте ansible на сервере BR-SRV

Для начала нам потребуется на BR-RTR, HQ-RTR и HQ-CLI настроить ssh по аналогии с HQ-SRV и BR-SRV с пятым заданием первого модуля. Для этого на BR-RTR и HQ-RTR открываем конфиг `/etc/openssh/sshd_config` и дописываем строки:

- Port 2026
- MaxAuthTries 2
- PermitRootLogin no
- AllowUsers net_admin

И перезагружаем ssh командой `systemctl restart sshd`.

На HQ-CLI создаем юзера ssh командой `/usr/sbin/useradd sshuser -u 2026`, после чего заданием ему пароль командой `echo "sshuser:P@ssw0rd" | chpasswd` и добавляем его в группу wheel командой `usermod -aG wheel sshuser`

```
hq-cli user # echo "sshuser:P@ssw0rd" | chpasswd
hq-cli user # /usr/sbin/usermod -aG wheel sshuser
hq-cli user #
```

Теперь открываем файл `/etc/sudoers` (либо просто пишем `visudo`) и дописываем или же раскомментируем строку `WHEEL_USERS ALL=(ALL:ALL)NOPASSWD:ALL` и также в `sshd_config` дописываем строки

- Port 2026
- MaxAuthTries 2
- PermitRootLogin no
- AllowUsers sshuser

Наконец, можем приступить к настройке самого ansible на BR-SRV. Для начала скачиваем пакет ansible командой `apt-get install ansible`. Потом открываем конфигурационный файл `/etc/ansible/hosts` и в блок `[clients]` (если его нет, создайте) впишем строки как на скриншоте:

```
GNU nano 8.0 /etc/ansible/hosts
[clients]
hq-rtr  ansible_host=192.168.1.1      ansible_user=net_admin  ansible_port=2026
br-rtr  ansible_host=192.168.4.1      ansible_user=net_admin  ansible_port=2026
hq-srv  ansible_host=192.168.1.2      ansible_user=sshuser    ansible_port=2026
hq_cli  ansible_host=192.168.2.6      ansible_user=sshuser    ansible_port=2026
```

Теперь нам нужно сгенерировать ssh ключи и раскидать их на всех клиентов. Для этого вводим команды:

- `ssh-keygen` (В процессе жмем Enter)
- `ssh-copy-id -p 2026 net_admin@192.168.1.1` // копируем ключ на hq-rtr
- `ssh-copy-id -p 2026 net_admin@192.168.4.1` // копируем ключ на br-tr
- `ssh-copy-id -p 2026 sshuser@192.168.1.2` // копируем ключ на hq-srv
- `ssh-copy-id -p 2026 sshuser@192.168.2.5` // копируем ключ на hq-cli

```

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh -p 2026 'net_admin@192.168.4.1'"
and check to make sure that only the key(s) you wanted were added.

[root@BR-SRV ~]# ssh-copy-id -p 2026 net_admin@192.168.1.2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_ed25519.pub"
The authenticity of host '[192.168.1.2]:2026 ([192.168.1.2]:2026)' can't be established.
ED25519 key fingerprint is SHA256:GCuuZucL9mUGpIDXyj+2rIdTSk8at3UCUFlgpU9+rBk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is t
authorized access only
net_admin@192.168.1.2's password:
ssh: Received disconnect from 192.168.1.2 port 2026:2: Too many authentication failures
Disconnected from 192.168.1.2 port 2026
[root@BR-SRV ~]# ssh-copy-id -p 2026 sshuser@192.168.1.2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is t
authorized access only
sshuser@192.168.1.2's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh -p 2026 'sshuser@192.168.1.2'"
and check to make sure that only the key(s) you wanted were added.

[root@BR-SRV ~]# ssh-copy-id -p 2026 sshuser@192.168.2.6
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_ed25519.pub"
The authenticity of host '[192.168.2.6]:2026 ([192.168.2.6]:2026)' can't be established.
ED25519 key fingerprint is SHA256:fd10b1J+0D+u0lkSrSSopb5ULE02Smq63DvJdgW6jGE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is t
sshuser@192.168.2.6's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh -p 2026 'sshuser@192.168.2.6'"
and check to make sure that only the key(s) you wanted were added.

[root@BR-SRV ~]#

```

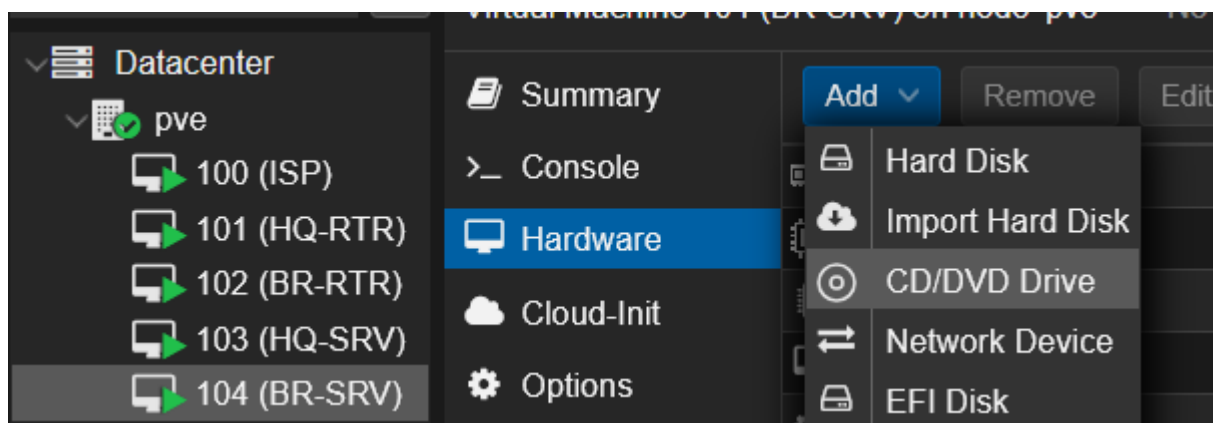
Теперь нам нужно настроить версию интерпретатора python для избежания конфликта версий, для этого открываем конфигурационный файл `/etc/ansible/ansible.cfg` и в блок `[default]` (если его нет, создайте) допишем строку `interpreter_python = python3`.

Сейчас можем проверить работоспособность ansible, выполнив в терминале команду `ansible all -m ping`

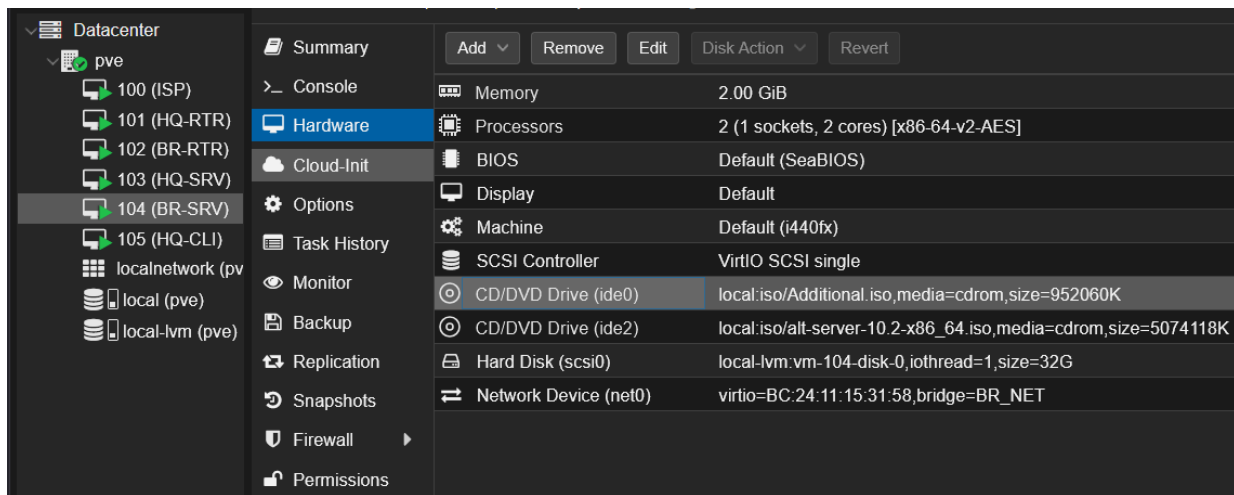

```
[root@BR-SRV ~]# ansible all -m ping
br-rtr | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
hq-rtr | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
hq-srv | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
hq_cli | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

6. Разверните веб приложение в docker на сервере BR-SRV

Первым делом установим пакеты docker compose и engine при помощи команды `apt-get install docker-engine docker-compose` и сразу пропишем `systemctl enable --now docker`. После этого, нам требуется подключить образ Additional.iso к виртуалке BR-SRV, для этого в Proxmox перейдем в пункт hardware BR-SRV, нажмем add и в появившемся меню выберем CD/DVD Drive



Там выберем хранилище local и образ Additional.iso и нажмем Add. После чего, перезагружаем виртуалку и диск отобразится все в том же пункте hardware



После этого на виртуалке введем `lsblk` и посмотрим доступные блочные устройства в системе.

```
[root@br-srv ~]# lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda          8:0    0    32G  0 disk
├─sda1       8:1    0   490M  0 part [SWAP]
└─sda2       8:2    0   31.5G  0 part /
sr0          11:0    1 929.7M  0 rom
sr1          11:1    1   4.8G  0 rom
```

Смотрим по размеру, какое именно устройство является нашим образом, в моем случае он называется `sr0`, в вашем случае может называться иначе. Когда мы узнали, как он называется, можем монтировать его в систему, перед этим создав директорию `/mnt/add_cd` командой `mkdir /mnt/add_cd`, после чего монтируем образ, введя команду `mount /dev/sr0 /mnt/add_cd` (если выдает ошибку, используйте `mount -t auto -o ro /dev/sr0 /mnt/add_cd`).

Теперь копируем из образа директорию докер в домашнюю директорию командой `cp -r /mnt/add_cd/docker ~/docker/` и сразу посмотрим содержимое этой директории командой `ls -lah docker/`

```
[root@BR-SRV ~]# cp -r /mnt/add_cd/docker/ ~/docker/
[root@BR-SRV ~]# ls -lah docker/
total 930M
dr-xr-xr-x  2 root root 4.0K Oct 23 23:12 .
drwx----- 11 root root 4.0K Oct 23 23:12 ..
-r-xr-xr-x  1 root root 318M Oct 23 23:12 mariadb_latest.tar
-r-xr-xr-x  1 root root 269M Oct 23 23:12 postgresql_latest.tar
-r-xr-xr-x  1 root root 2.7K Oct 23 23:12 readme.txt
-r-xr-xr-x  1 root root 344M Oct 23 23:12 site_latest.tar
```

Нам потребуются архивы `site_latest.tar` и образ субд `mariadb_tar`. Импортируем их в докер при помощи команд `docker image load -i /root/docker/site_latest.tar` и `docker image load -i /root/docker/mariadb_latest.tar`

```
[root@BR-SRV ~]# docker image load -i /root/docker/site_latest.tar
418dceb7d85a: Loading layer [=====>] 8.596MB/8.596MB
009e09bc9ce7: Loading layer [=====>] 1.676MB/1.676MB
ab5dfdbd0211: Loading layer [=====>] 43.22MB/43.22MB
16dad80c55d7: Loading layer [=====>] 5.12kB/5.12kB
2a6ab270dfc4: Loading layer [=====>] 1.536kB/1.536kB
f296fa7d3e68: Loading layer [=====>] 176.4MB/176.4MB
c7d44a5452c6: Loading layer [=====>] 3.072kB/3.072kB
5a221749f04b: Loading layer [=====>] 129MB/129MB
b737b3a43bb8: Loading layer [=====>] 836.1kB/836.1kB
Loaded image: site:latest
[root@BR-SRV ~]# docker image load -i /root/docker/mariadb_latest.tar
767e56ba346a: Loading layer [=====>] 80.42MB/80.42MB
f30f0fd94399: Loading layer [=====>] 337.9kB/337.9kB
29698e3a4235: Loading layer [=====>] 16.33MB/16.33MB
1a569e551418: Loading layer [=====>] 1.536kB/1.536kB
63e39b3459b1: Loading layer [=====>] 5.12kB/5.12kB
599b2700498c: Loading layer [=====>] 235.0MB/235.0MB
adf17a73d7f3: Loading layer [=====>] 13.82kB/13.82kB
64b59101975d: Loading layer [=====>] 29.7kB/29.7kB
Loaded image: mariadb:10.11
```

После чего смотрим чего мы там наимпортировали командой `docker images`

```
[root@BR-SRV ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
site                 latest             015b4b821098       2 weeks ago        353MB
mariadb              10.11             bc52d24721da       2 months ago       327MB
```

Теперь создаем директорию `testapp` при помощи командой `mkdir testapp` и перейдем в нее командой `cd testapp`. В ней создаем конфигурационный файл `docker-compose.yaml` и заполняем по подобию:

```
GNU nano 8.0
services:
  testapp:
    image: site:latest
    container_name: testapp
    restart: always
    depends_on:
      - db
    ports:
      - 8080:8080
    environment:
      DB_TYPE: maria
      DB_HOST: db
      DB_NAME: testdb
      DB_PORT: 3306
      DB_USER: testc
      DB_PASS: Passw0rd
  db:
    image: mariadb:latest
    container_name: db
    restart: always
    environment:
      DB_NAME: testdb
      DB_USER: testc
      DB_PASS: Passw0rd
      MARIADB_ROOT_PASSWORD: 123qweR%
    volumes:
      - /root/testapp/db_data:/var/lib/mysql
volumes:
  db_data:
```

*Будьте внимательны, поскольку это файл yaml, тут очень важны отступы, они должны быть точь-в-точь как на скриншоте.

Теперь запускаем стек контейнеров `docker compose up -d` и смотрим список запущенных контейнеров командой `docker ps`

```
[root@DB-SRV testapp]# docker compose up -d
[*] Running 9/9
[+] db Pulled
  4b3ffd8ccb52 Pull complete
  7854007237e1 Pull complete
  8ed1b5271813 Pull complete
  402f4c7d4065 Pull complete
  2295facceabc Pull complete
  eece26ecda4b Pull complete
  4c2e197f6eb9 Pull complete
  6499e85d8558 Pull complete
[*] Running 3/3
[+] Network testapp_default Created
[+] Container db Started
[+] Container testapp Started
[root@DB-SRV testapp]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c30c57c06213	site:latest	"sh -c 'python3 -n a..."	13 seconds ago	Up 11 seconds	0.0.0.0:8080->8080/tcp	testapp
bbe117c44391	mariadb:latest	"docker-entrypoint.s..."	14 seconds ago	Up 13 seconds	3306/tcp	db

Далее придется чуть-чуть вспоминать базы данных 🙄. Заходим в контейнер db и запускаем субд командой `docker exec -it db mysql -u root -p` (команда может отличаться в зависимости от версий, например, в моем случае она выглядела так: `docker run -it --rm --network testapp_default mariadb:10.11 mysql -h db -u root -p`, у вас может иметь

другой вид, ищите сами), после чего вводим пароль 123qweR%. Теперь выполняем следующие команды:

```
CREATE DATABASE testdb; // создание БД
```

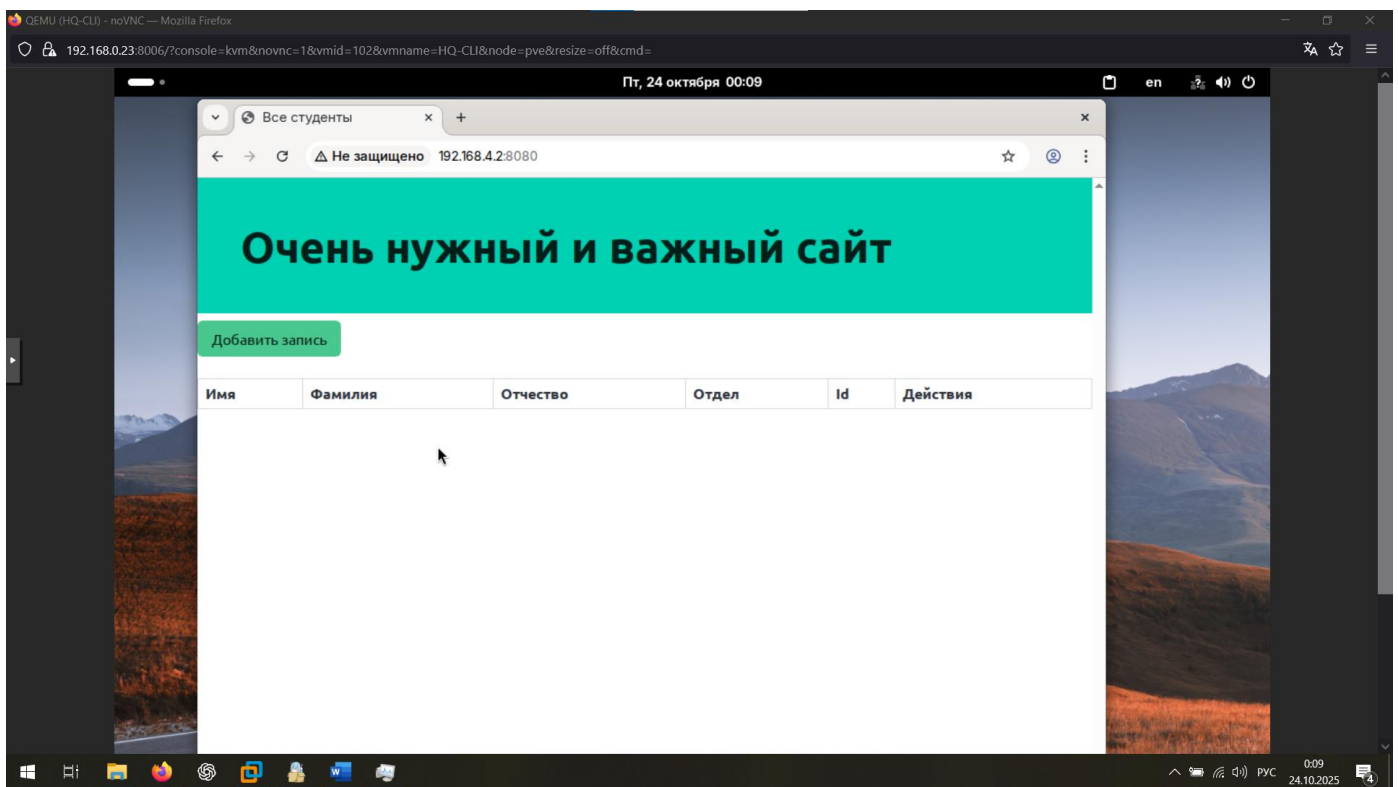
```
CREATE USER 'testc'@'%' IDENTIFIED BY 'Passw0rd'; // создание юзера
```

```
GRANT ALL PRIVILEGES ON testdb.* TO 'testc'@'%'; // выдаем пользователю права на бд
```

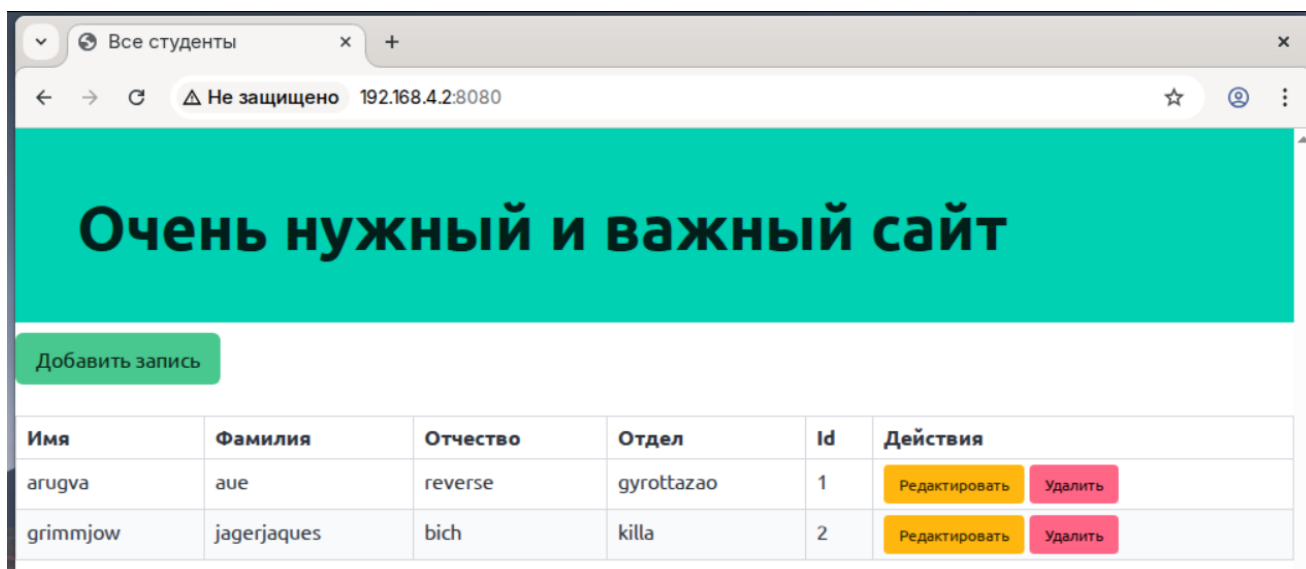
```
FLUSH PRIVILEGES; // применяем изменения
```

```
exit // выходим
```

Проверяем работоспособность. Для этого переходим на HQ-CLI, открываем браузер и вводим в поисковую строку айпи адрес BR-SRV с указанием порта 8080, хвала богам, мы попали на сайт



Для теста попробуем внести пару записей и обновить сайт



7. Разверните веб приложение на сервере HQ-SRV

Первым делом нам требуется установить кучу пакетов, а конкретно apache2 mariadb php8.2 apache2-mod_php8.2 php8.2-mysqli

```
[root@hq-srv ~]# apt-get install apache2 mariadb php8.2 apache2-mod_php8.2 php8.2-mysqli
Reading Package Lists... Done
Building Dependency Tree... Done
```

После установки всех пакетов, добавим службы httpd2 и mariadb в автозагрузку командами `systemctl enable --now httpd2` и `systemctl enable --now mariadb`.

Сейчас подключаем по такой же схеме как в прошлой задании подключаем образ с сайтом. После того, как добавили его в Proxmox, вводим команду `lsblk` и определяем его имя. Когда узнали имя, можем создать директорию для монтирования командой `mkdir -p /mnt/add_cd`. После этого вводим команду `mount -t auto -o ro /dev/sr0 /mnt/add_cd`.

```
[root@hq-srv ~]# mkdir -p /mnt/add_cd
[root@hq-srv ~]# mount -t auto -o ro /dev/sr0 /mnt/add_cd/
[root@hq-srv ~]# ls /mnt/add_cd/
Users.csv  docker  playbook  web
```

Теперь вводим команду `cp -r /mnt/add_cd/web /root/web`, чтобы копировать содержимое `/mnt/add_cd/web` в директорию `/root/web`.

Сейчас можем приступить к инициализации базы данных, для этого введем команду `mariadb_secure_installation`. В процессе установки нам будут задавать вопросы, на все отвечаем Y и один раз введем пароль для администратора P@ssw0rd.

```

[root@hq-srv ~]# mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

Enable unix_socket authentication? [Y/n] y
Enabled successfully!
Reloading privilege tables..
... Success!

You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n] y
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] y
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y_

```

Далее вводим команду `mysql` что бы войти в систему управления базой данных, там вводим команду `CREATE DATABASE webdb;` что бы создать базу данных.

```

[root@hq-srv ~]# mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 13
Server version: 10.6.23-MariaDB-alt1 (ALT p10)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE webdb;
Query OK, 1 row affected (0.007 sec)

MariaDB [(none)]> exit
Bye

```

Когда база данных создана, нам требуется импортировать в нее дамп из директории `/web/`, которую мы копировали ранее. Что бы это сделать, введем в терминал команду `mysql webdb < /root/web/dump.sql`.

```

[root@hq-srv ~]# mysql webdb < /root/web/dump.sql

```


После этого входим в субд при помощи команды `mysql` и вводим команды:

```
CREATE USER 'webc'@'%' IDENTIFIED BY 'P@ssw0rd'; // Создаем пользователя
GRANT ALL PRIVILEGES ON webdb.* TO 'webc'@'%'; // Выдаем все права в базе
webdb пользователю webc
FLUSH PRIVILEGES; // Применяем изменения
Exit // Выходим из субд
```

После этого открываем файл `/root/web/index.php` и приводим первые 5 строк к следующему виду:

```
<?php
$servername = "localhost";
$username = "webc";
$password = "P@ssw0rd";
$dbname = "webdb";
```

Теперь нам нужно скопировать `Index.php` и `logo.png` в директорию `/var/www/html/`. Для этого выполняем команды `cp /root/web/index.php /var/www/html` и `cp /root/web/logo.png /var/www/html/logo.png`, а также удаляем из директории `/var/www/html` файл `index.html` командой `rm -f /var/www/html/index.html`

```
[root@hq-srv ~]# cp /root/web/index.php /var/www/html
cp: overwrite '/var/www/html/index.php'? yes
[root@hq-srv ~]# cp /root/web/logo.png /var/www/html/logo.png
[root@hq-srv ~]# rm -f /var/www/html/index.html
```

В итоге директория `/var/www/html` должна выглядеть следующим образом

```
[root@hq-srv ~]# tree /var/www/html
/var/www/html
├── addon-modules
├── index.php
└── logo.png
```

Что бы арасче мог работать с `php8.2` нам требуется включить модуль `mod_php8.2` при помощи команды `a2enmod mod_php8.2` и перезагрузить службу `httpd2` командой `systemctl restart httpd2`

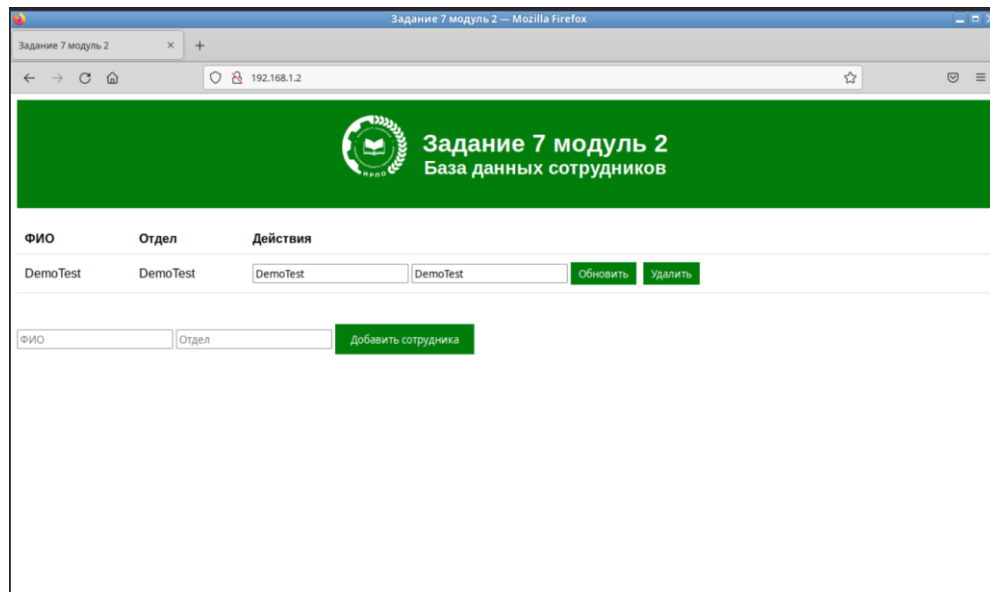
```

[root@hq-srv ~]# a2enmod mod_php8.2
Module mod_php8.2 is already enabled!
Module mod_php8.2 installed;
        run service httpd2 condreload to fully enable.
[root@hq-srv ~]# systemctl restart httpd2
[root@hq-srv ~]#

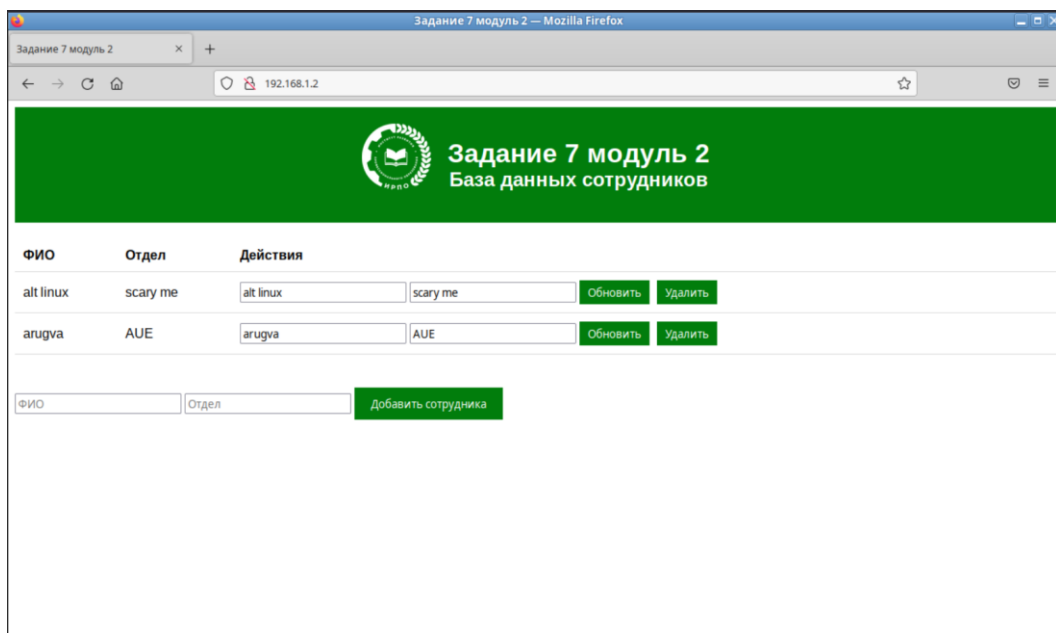
```

Теперь apache сможет исполнять PHP скрипты версии 8.2

Настройка сервера завершена, теперь с HQ-CLI пробуем зайти на сайт. Для этого в поисковой строке браузера введем адрес HQ-SRV



Для теста попробуем добавить пару записей и обновить сайт



Записи сохранились после обновления сайта, а значит все работает отлично.

8. На маршрутизаторах сконфигурируйте статическую трансляцию портов

Нам требуется выполнить проброс портов на HQ-RTR и BR-RTR, чтобы веб приложения и ssh на серверах были доступны извне. Для выполнения этого задания воспользуемся крутой утилитой iptables.

8.1 Пробросьте порт 8080 в порт приложения testapp BR-SRV на маршрутизаторе BR-RTR, для обеспечения работы приложения testapp извне.

В консоль на BR-RTR вводим следующее правило iptables:

```
Iptables -t nat -A PREROUTING -d 172.16.5.2 -p tcp --dport 8080 -j DNAT --to-destination 192.168.4.2:8080
```

```
root@br-rtr:/home/user# iptables -t nat -A PREROUTING -d 172.16.5.2 -p tcp --dport 8080 -j DNAT --to-destination 192.168.4.2:8080
root@br-rtr:/home/user#
```

8.2 Пробросьте порт 2026 на маршрутизаторе BR-RTR в порт 2026 сервера BR-SRV, для подключения к серверу по протоколу ssh из внешних сетей.

На BR-RTR выполним вводим то же самое правило, что и предыдущее, но изменим порт с 8080 на 2026:

```
Iptables -t nat -A PREROUTING -d 172.16.5.2 -p tcp --dport 2026 -j DNAT --to-destination 192.168.4.2:2026
```

```
root@br-rtr:/home/user# iptables -t nat -A PREROUTING -d 172.16.5.2 -p tcp --dport 2026 -j DNAT --to-destination 192.168.4.2:2026
root@br-rtr:/home/user#
```

В сторону BR настройка завершена, можем посмотреть добавленные правила введя команду `iptables -L -n -v -t nat`

```
root@br-rtr:/home/user# iptables -v -n -L -t nat
Chain PREROUTING (policy ACCEPT 1 packets, 328 bytes)
 pkts bytes target    prot opt in     out     source    destination
    0      0 DNAT      tcp  --  *      *       0.0.0.0/0  172.16.5.2      tcp dpt:8080 to:192.168.4.2:8080
    0      0 DNAT      tcp  --  *      *       0.0.0.0/0  172.16.5.2      tcp dpt:2026 to:192.168.4.2:2026
```

Можем увидеть, что правила появились в цепочке PREROUTING, как и должно быть.

Также, если вы не хотите потерять эти правила при перезагрузке устройства, сохраните их в файл командой `iptables-save > /etc/iptables.rules`

8.3 Пробросьте порт 8080 в порт веб приложения на HQ-SRV на маршрутизаторе HQ-RTR, для обеспечения работы веб приложения извне

Для этого на HQ-RTR введем в консоль команду:

`Iptables -t nat -A PREROUTING -d 172.16.4.2 -p tcp --dport 8080 -j DNAT --to-destination 192.168.1.2:8080`

```
root@hq-rtr:/home/user# iptables -t nat -A PREROUTING -d 172.16.4.2 -p tcp --dport 8080 -j DNAT --to-destination 192.168.1.2:8080
root@hq-rtr:/home/user# _
```

8.4. Пробросьте порт 2026 на маршрутизаторе HQ-RTR в порт 2026 сервера HQ-SRV, для подключения к серверу по протоколу ssh из внешних сетей

Для выполнения этого задания в консоль HQ-RTR следующее правило iptables:

`Iptables -t nat -A PREROUTING -d 172.16.4.2 -p tcp --dport 2026 -j DNAT --to-destination 192.168.1.2:2026`

```
root@hq-rtr:/home/user# iptables -t nat -A PREROUTING -d 172.16.4.2 -p tcp --dport 2026 -j DNAT --to-destination 192.168.1.2:2026
root@hq-rtr:/home/user#
```

Теперь проверим добавленные правила, введя команду `iptables -n -v -L -t nat`

```
root@hq-rtr:/home/user# iptables -n -v -L -t nat
Chain PREROUTING (policy ACCEPT 791 packets, 54283 bytes)
pkts bytes target      prot opt in     out    source    destination
 0      0 DNAT        tcp  --  *      *      0.0.0.0/0 172.16.4.2      tcp dpt:8080 to:192.168.1.2:8080
 0      0 DNAT        tcp  --  *      *      0.0.0.0/0 172.16.4.2      tcp dpt:2026 to:192.168.1.2:2026
```

Также сохраним правила iptables при помощи команды `iptables-save > /etc/iptables.rules`

На этом выполнение задания окончено.

9. Настройте веб-сервер nginx как обратный прокси-сервер на ISP.

Reverse proxy – принимает запросы от клиентов и отправляет их на нужные серверы. Нам требуется сделать так, чтобы при обращении к ISP по доменным именам (web и docker) у нас открывались соответствующие сайты.

Для выполнения этого задания нам потребуется установить nginx командой `apt-get install nginx`.

После установки открываем конфигурационный файл по пути `/etc/nginx/sites-available/default` (либо `/etc/nginx/sites-available.d/default.conf`, если есть) и заполняем по следующему подобию:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    root /var/www/html;

    server_name web.au-team.irpo;

    location / {
        root /var/www/html;
        proxy_pass http://172.16.4.2:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarder-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarder-Proto $scheme;
    }
}

server {
    listen 80;
    server_name docker.au-team.irpo;
    location / {
        proxy_pass http://172.16.5.2:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarder-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
server{
```

```
    listen 80; //слушает порт 80
```

```
    server_name docker.au-team.irpo; //домен для которого работает
```

```
    location / { //обрабатывает все запросы к корню
```

```
        proxy_pass http://172.16.5.2:8080; // перенаправляет запросы на сервер
```

```
proxy_set_header Host $host; //передает оригинальный домен
```

```
proxy_set_header X-Real-IP $remote_addr; //реальный IP клиента
```

```
proxy_set_header X-Forwarder-For $proxy_add_x_forwarded_for; //цепочка прокси
```

```
proxy_set_header X-Forwarded-Proto $scheme; //протокол
```

***! На самом деле, оказалось, что в конфиге достаточно строк**

```
server{
```

```
    listen 80;
```

```
    server_name docker.au-team.irpo;
```

```
location / {
```

```
    proxy_pass http://172.16.5.2:8080; и
```

```
server{
```

```
    listen 80;
```

```
    server_name web.au-team.irpo;
```

```
location / {
```

```
    proxy_pass http://172.16.4.2:8080
```

Остальное пишется по желанию)

После заполнения конфигурации проверим конфиг на синтаксические ошибки, для этого выполним встроенную в nginx команду `nginx -t`

```
root@isp:/home/user# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Ошибок нет, все отлично, перезагружаем службу командой `systemctl restart nginx`.

Теперь нам нужно создать символическую ссылку в директории `/etc/nginx/sites-enabled` на файл с прокси nginx. Для этого выполняем команду `ln -s /etc/nginx/sites-`

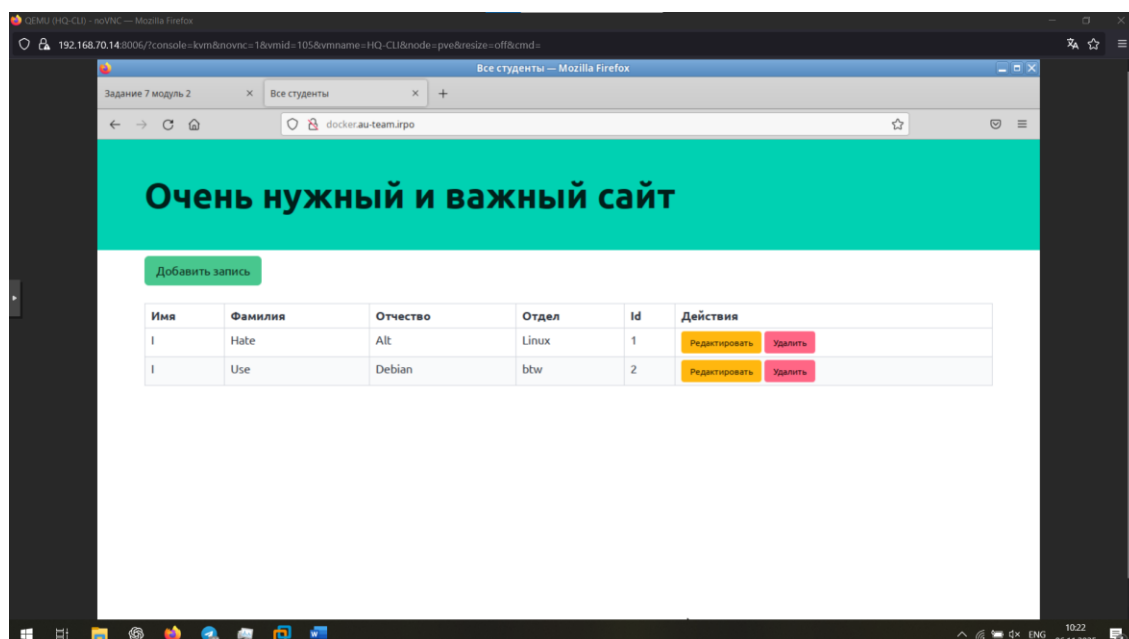
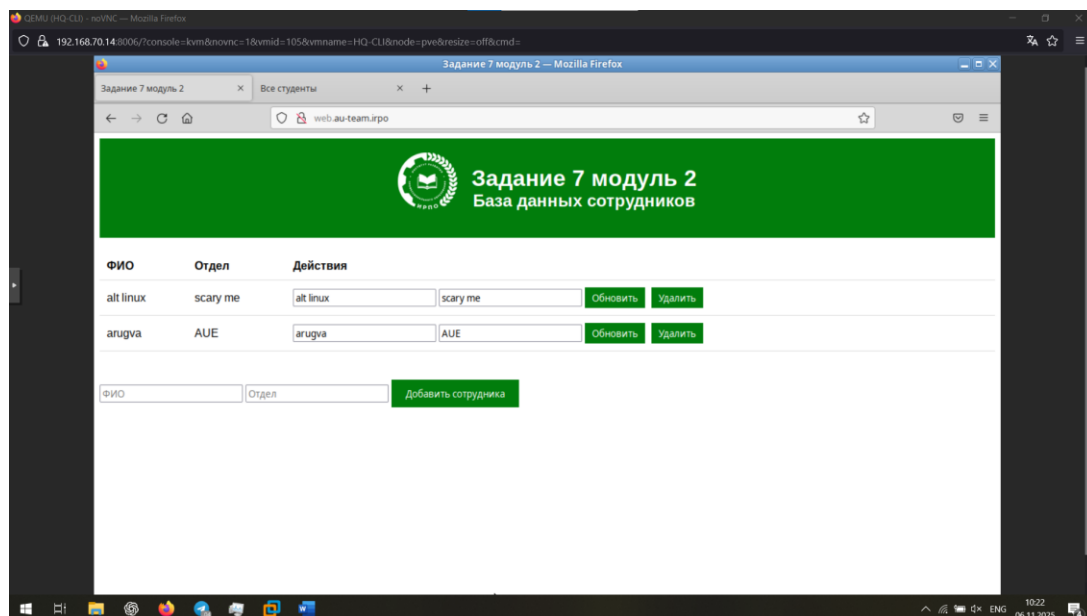
available/default /etc/nginx/sites-enabled/default (Если выдаст ошибку, что файл уже существует, все нормально, значит конфиг каким-то чудом заполнился сам).

Остался последний шаг перед проверкой работоспособности. Переходим на HQ-SRV, идем в конфиг /etc/httpd2/conf/ports-enabled/http.conf и ставим LISTEN 8080

```
#  
#Listen 12.34.56.78:  
Listen 8080_
```

После чего перезагружаем apache командой systemctl restart httpd2.

Сейчас можем переходить на HQ-CLI, открываться в браузер и вводить в поисковую строку адреса http://web.au-team.irpo и http://docker.au-team.irpo



Поздравляю, у нас все работает. Двигаемся далее.

10. На маршрутизаторе ISP настройте web-based аутентификацию

В этом задании нам нужно настроить web-аутентификацию для web.au-team.irpo. Для этого на ISP установим пакет apache2-utils командой `apt-get install apache2-utils`.

После установки создаем пользователя WEBc командой `htpasswd -c /etc/nginx/.htpasswd WEBc` и два раза вводи пароль P@ssw0rd

```
root@isp:/home/user# htpasswd -c /etc/nginx/.htpasswd WEBc
New password:
Re-type new password:
Adding password for user WEBc
```

Теперь открываем конфиг `/etc/nginx/sites-available/default` и в блок с сервером web.au-team.irpo дописываем две строки:

- `auth_basic "Web-authorization";`
- `auth_basic_user_file /etc/nginx/.htpasswd;`

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

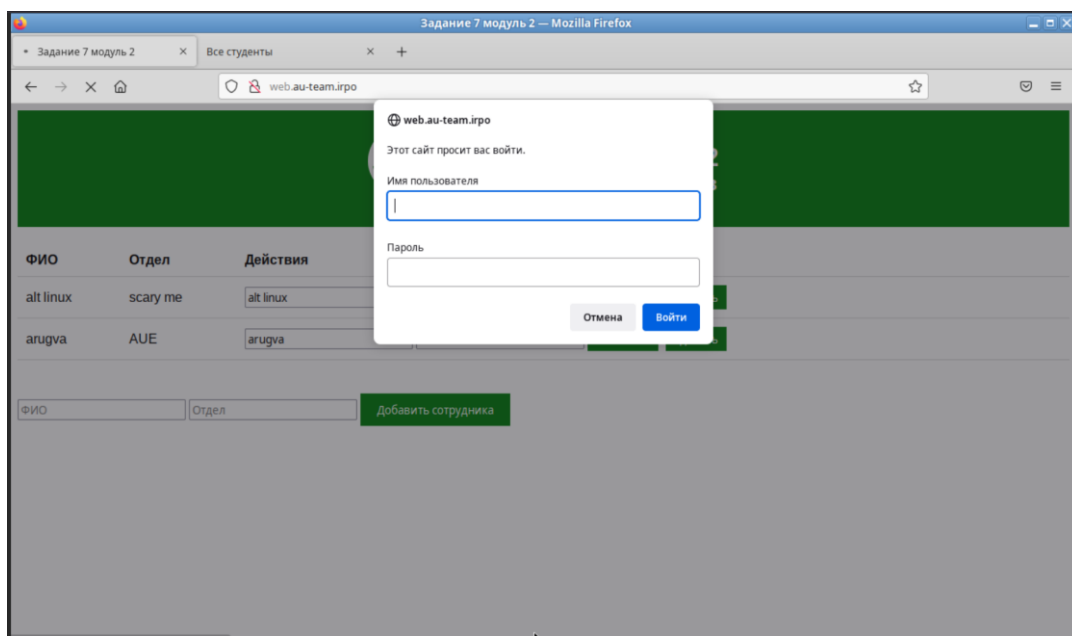
    root /var/www/html;

    server_name web.au-team.irpo;

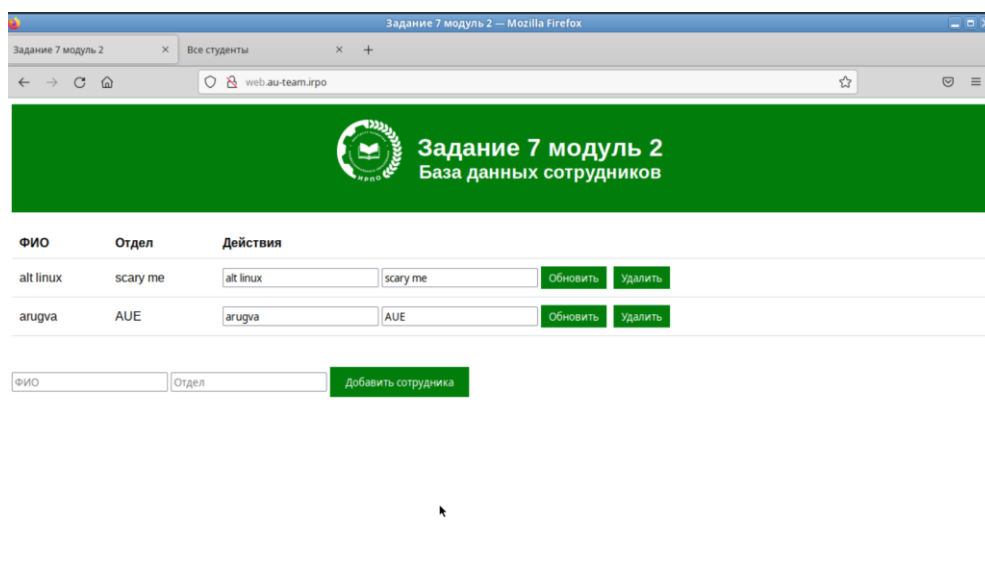
    location / {
        root /var/www/html;
        proxy_pass http://172.16.4.2:8080;
        auth_basic "Web-authorization";
        auth_basic_user_file /etc/nginx/.htpasswd;
```

После чего перезапускаем nginx командой `systemctl restart nginx`.

Теперь можем проверить авторизацию. Для этого с HQ-CLI зайдём на сайт web.au-team.irpo



Сразу при входе на сайт у нас появляется окошко для ввода логина и пароля.
Вводим логин WEBc и пароль P@ssw0rd



После этого нас пускает на сайт.

ДАЛЬШЕ ВЫПОЛНЯЕМ САМОЕ СЛОЖНОЕ ЗАДАНИЕ ВСЕГО
ДЕМОЭКЗАМЕНА.

11. Удобным способом установите приложение Яндекс Браузер на HQ-CLI

В консоли на HQ-CLI вводим команду `apt-get install yandex-browser-stable`

```
user@hq-cli: /home/user
Файл Правка Вид Поиск Терминал Помощь
user@hq-cli ~ $ su
Password:
hq-cli ~ # apt-get install yandex-browser-stable
Чтение списков пакетов... Завершено
Построение дерева зависимостей... Завершено
Следующие дополнительные пакеты будут установлены:
  ffmpeg-plugin-browser fonts-ttf-google-noto-emoji-color
Следующие НОВЫЕ пакеты будут установлены:
  ffmpeg-plugin-browser fonts-ttf-google-noto-emoji-color
  yandex-browser-stable
0 будет обновлено, 3 новых установлено, 0 пакетов будет удалено и 681 не будет о
бновлено.
Необходимо получить 229MB архивов.
После распаковки потребуется дополнительно 602MB дискового пространства.
Продолжить? [Y/n]
```

Нажимаем у и ждем....

```
Совершаем изменения...
Подготовка... ##### [100%]
Обновление / установка...
1: fonts-ttf-google-noto-emoji-color-1:2##### [ 33%]
2: ffmpeg-plugin-browser-132-alt1 ##### [ 67%]
3: yandex-browser-stable-25.2.4.1000-alt##### [100%]
Завершено.
hq-cli ~ #
```

После утомительного ожидания, мы наконец то можем открыть наш любимый, великий и неподражаемый Яндекс Браузер командой `yandex-browser-stable --no-sandbox`

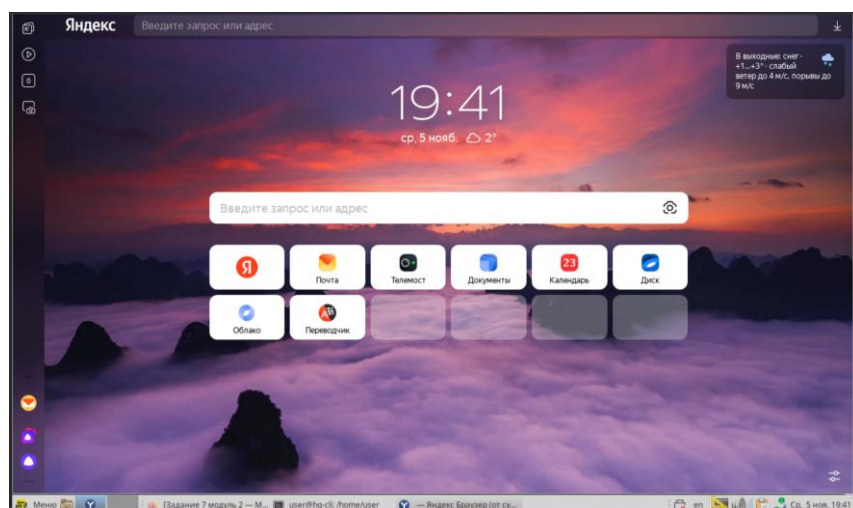
```

Ffmpeg version is OK! Let's use it.
[13880:13880:1105/193916.255989:ERROR:corp_license_service_factory.cc(115)] [CORP] License not found
[13880:13880:1105/193919.038287:ERROR:isolated_origin_util.cc(75)] Ignoring port number in isolated origin: chrome://custo
[13880:13880:1105/193919.041662:ERROR:child_process_security_policy_impl.cc(2569)] Invalid isolated origin:
[13880:13911:1105/193919.080313:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13911:1105/193919.101852:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13911:1105/193919.133897:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13911:1105/193919.176511:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13911:1105/193919.177596:ERROR:object_proxy.cc(576)] Failed to call method: org.freedesktop.DBus.NameHasOwner: object_path= /org/freedesktop/DBus: unknown error type:
[13928:13928:1105/193920.411573:ERROR:viz_main_impl.cc(191)] Exiting GPU process due to errors during initialization
[13880:13880:1105/193923.122294:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13911:1105/193923.176401:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13880:1105/193923.608407:ERROR:anti_tracking_service.cc(163)] [CORP] YandexAntiTracking_Status: Политика отключена.
[13880:13911:1105/193924.736693:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13911:1105/193924.737338:ERROR:object_proxy.cc(576)] Failed to call method: org.freedesktop.DBus.NameHasOwner: object_path= /org/freedesktop/DBus: unknown error type:
[13880:13880:1105/193927.428518:ERROR:object_proxy.cc(576)] Failed to call method: org.freedesktop.DBus.NameHasOwner: object_path= /org/freedesktop/DBus: unknown error type:
[13880:13880:1105/193932.417695:ERROR:fallback_task_provider.cc(126)] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/739782.
XDG_RUNTIME_DIR (/run/user/500) принадлежит не данному пользователю (uid 0), а пользователю с uid 500. (Это может происходить, например, в случае подключения от имени администратора к серверу PulseAudio, запущенному от имени обычного пользователя, по родному протоколу. Не делайте так.)
[13975:13975:1105/193936.311369:ERROR:viz_main_impl.cc(191)] Exiting GPU process due to errors during initialization
[13880:13911:1105/193947.116380:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13911:1105/193947.128864:ERROR:bus.cc(407)] Failed to connect to the bus: Did not receive a reply. Possible causes include: the remote application did not send a reply, the message bus security policy blocked the reply, the reply timeout expired, or the network connection was broken.
[13880:13880:1105/193956.179461:ERROR:fallback_task_provider.cc(126)] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/739782.
[13880:13880:1105/194000.488771:ERROR:fallback_task_provider.cc(126)] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/739782.
[13880:13880:1105/194009.386813:ERROR:fallback_task_provider.cc(126)] Every renderer should have at least one task provided by a primary task provider. If a "Renderer" fallback task is shown, it is a bug. If you have repro steps, please file a new bug and tag it as a dependency of crbug.com/739782.

```

Забавно, что в консоли при этом куча ошибок))

Лицезреем Яндекс



На этом второй модуль завершен.

3 МОДУЛЬ

1. Выполните импорт пользователей в домен au-team.irpo

На этом задании у меня возникли большие вопросы к тем, кто его составлял, поскольку нет буквально ни одного нормального человеческого способа импортировать пользователей в samba dc. Лично у меня было всего 2 варианта:

добавлять всех пользователей вручную, либо написать bash-скрипт, который будет добавлять пользователей. В итоге мой выбор, конечно же, пал на второй вариант, ведь если добавлять такое количество пользователей вручную, можно сойти с ума.

Сначала, скопируем файл с пользователями в директорию /root/. Для этого вводим команду `cp /mnt/add_cd/Users.csv /root/etc/users.csv`

```
[root@br-srv ~]# cp /mnt/add_cd/Users.csv /root/users.csv
```

После этого создадим и откроем в текстовом редакторе скрипт `/root/import.sh` и заполним следующим образом:

```
#!/bin/bash
CSV="/root/users.csv"
DOMAIN="au-team.irpo"
DEFAULT_PASS="P0ssw0rd"
while IFS=',' read -r fname lname title phone dept street house zip city country password; do
    [[ "$fname" == "Name" || -z "$fname" ]] && continue
    fname=$(echo "$fname" | xargs)
    lname=$(echo "$lname" | xargs)

    username=$(echo "${fname,,}.${lname,,}" | tr -d ' ')
    displayname="$fname $lname"
    echo "Create user: $displayname ($username)"
    samba-tool user create "$username" "$DEFAULT_PASS" \
        --userou="OU=Users" \
        --given-name="$fname" \
        --surname="$lname" \
        --description="$title" \
        --home-drive="H:" \
        --script-path="logon.bat" \
    done < "$CSV"
    echo "Import finished succesfully"
```

*Я не совсем уверен, что этот скрипт полностью соответствует заданию и файлу `users.csv`, однако на большее меня не хватило.

После этого нам нужно запустить скрипт. Для этого перейдем в директорию `root` командой `cd /root/` и запустим скрипт `./import.sh`

```
[root@br-srv ~]# ./import.sh
Create user: First Name Last Name (first.name.last.name)
WARNING: Using passwords on command line is insecure. Installing the setproctitle python module will hide these from shortly after program start.
User 'first.name.last.name' added successfully
Create user: Malachi Alexander (malachi.alexander)
WARNING: Using passwords on command line is insecure. Installing the setproctitle python module will hide these from shortly after program start.
User 'malachi.alexander' added successfully
Create user: Nelle Alford (nelle.alford)
WARNING: Using passwords on command line is insecure. Installing the setproctitle python module will hide these from shortly after program start.
User 'nelle.alford' added successfully
Create user: Meredith Arnold (meredith.arnold)
WARNING: Using passwords on command line is insecure. Installing the setproctitle python module will hide these from shortly after program start.
User 'meredith.arnold' added successfully
Create user: Hasad Ashley (hasad.ashley)
WARNING: Using passwords on command line is insecure. Installing the setproctitle python module will hide these from shortly after program start.
```


У нас начинают добавляться пользователи. Это надолго, ожидание может занять до 30 минут.

После мучительного ожидания мы увидим вывод «import finished succesfully»

```
Create user: Stuart York (stuart.york)
WARNING: Using passwords on command line is insecure. Installing the setproctitle python module
User 'stuart.york' added successfully
Import finished succesfully
```

Теперь проверим, корректно ли у нас добавились пользователи, для этого выполним команду `samba-tool user list`

```
rudyard.house
nissim.webb
cyrus.gomez
gabriel.mcdowell
meredith.arnold
hasad.ashley
nolan.barry
stephen.willis
noah.buck
adria.logan
hayes.stokes
genevieve.vega
macey.waters
nash.hood
emily.schroeder
lucian.buck
abbot.blackwell
emmanuel.blankenship
helen.nichols
raphael.gaines
amber.roth
leigh.torres
samantha.wilkins
lois.bruce
nissim.blanchard
aspen.terrell
hunter.hopkins
carolyn.parsons
patricia.lucas
anastasia.faulkner
amery.boyer
lydia.sears
jessamine.hodge
pearl.davidson
rigel.tyson
reuben.sanford
jermaine.simmons
fay.gallegos
roary.conley
alika.barron
karly.curry
james.macias
kelsie.horton
addison.wall
wyatt.fleming
```

У нас вывелся огромный список с кучей пользователей, значит скрипт отработал так, как надо.

2. Выполните настройку центра сертификации на базе HQ-SRV.

Эту дичь я не вывез 😄 😄 😄

3. Перенастройте ip-туннель с базового до уровня туннеля, обеспечивающего шифрование

Инсталлируем на оба роутера пакеты strongswan, включающие ipsec.

Открываем конфиг `/etc/ipsec.conf` и приводим к следующему виду:

На HQ-RTR:

```
# basic configuration

config setup
    charondebug="ike 1, knl 1, cfg 0"
    uniqueids=yes

conn hq-br
    left=172.16.4.2
    leftid=172.16.4.2
    right=172.16.5.2
    rightid=172.16.5.2
    auto=start
    type=transport
    keyexchange=ikev2
    ike=aes256-sha256-modp2048
    esp=aes256-sha256
    authby=psk
    auto = start
    mobike=no
```

Где:

- `config setup` - начало секции базовых настроек;
- `charondebug='like 1, knl 1, cfg 0'` - уровень отладки разных компонентов;
- `uniqueids=yes` - использование уникальных идентификаторов для подключений;
- `conn hq-br` - определение подключения с именем "hq-br";
- `left=172.16.4.2` - локальный IP-адрес;
- `leftid=172.16.4.2` - идентификатор локальной стороны;
- `right=172.16.5.2` - удаленный IP-адрес;
- `rightid=172.16.5.2` - идентификатор удаленной стороны;
- `auto=start` - автоматический запуск подключения;

- type=transport - транспортный режим (без инкапсуляции);
- keyexchange=ikev2 - использование IKEv2 протокола;
- ike=aes256-sha256-modp2048 - параметры IKE: AES256, SHA256, группа DH 2048;
- esp=aes256-sha256 - параметры ESP: AES256, SHA256;
- authby=psk - аутентификация с помощью предварительного ключа;
- mobike=no - отключение функции MOBIKE (может сильно помешать).

На BR-RTR:

```
config setup
    charondebug="ike 1, knl 1, cfg 0"
    uniqueids=yes

conn br-hq
    left=172.16.5.2
    leftid=172.16.5.2
    leftsubnet=192.168.4.0/27
    right=172.16.4.2
    rightid=172.16.4.2
    rightsubnet=192.168.1.0/27,192.168.2.0/28
    auto=start
    type=tunnel
    keyexchange=ikev2
    ike=aes256-sha256-modp2048!
    esp=aes256-sha256!
    authby=secret
```

После чего на обоих роутерах в конфиг `/etc/ipsec.secrets` добавляем строку

172.16.4.2 172.16.5.2 : PSK "P@ssw0rd"

```
include /var/lib/strongswan/ipsec.secrets
172.16.4.2 172.16.5.2 : PSK "P@ssw0rd"
```

И перезагружаем strongswan командой `systemctl restart strongswan`.

Активируем подключение командой `ipsec up hq-br`

```
root@hq-rtr:/home/user# ipsec up hq-br
establishing CHILD_SA hq-br
generating CREATE_CHILD_SA request 0 [ SA No TSi TSr ]
sending packet: from 172.16.4.2[4500] to 172.16.5.2[4500] (224 bytes)
received packet: from 172.16.5.2[4500] to 172.16.4.2[4500] (224 bytes)
parsed CREATE_CHILD_SA response 0 [ SA No TSi TSr ]
connection 'hq-br' established successfully
```

Должно вывести established successfully.

Теперь в несколько шагов проверим, реально ли работает защищенный туннель.

Сперва вводим команду `ip xfrm state`, вывод должен быть примерно таким:

```
root@hq-rtr:/home/user# ip xfrm state
src 172.16.4.2 dst 172.16.5.2
  proto esp spi 0xc2bcebad reqid 1 mode transport
  replay-window 0
  auth-trunc hmac(sha256) 0x0bd536d4e18e6617d0c2ed098a52e43b2fcbf46e169a3ee821c41d4b04d74a1a 128
  enc cbc(aes) 0x813d392774e36f91e34007bd373be7c2fc688344f31f9ec2d8efe888aef57ceb
  anti-replay context: seq 0x0, oseq 0x2c7, bitmap 0x00000000
  sel src 172.16.4.2/32 dst 172.16.5.2/32
src 172.16.5.2 dst 172.16.4.2
  proto esp spi 0xc304c6c9 reqid 1 mode transport
  replay-window 32
  auth-trunc hmac(sha256) 0xd2a41b0afb351ef68dd4ec00e68541be970dbec42694c69bbac88c6c26b8815 128
  enc cbc(aes) 0x03f1552a95d83e728932e64ed6514fd86273367ccdf0b6ffa0379b10ef90d442
  anti-replay context: seq 0x2c8, oseq 0x0, bitmap 0xff7fefff
  sel src 172.16.5.2/32 dst 172.16.4.2/32
src 172.16.4.2 dst 172.16.5.2
  proto esp spi 0xc6d8ccbe reqid 1 mode transport
  replay-window 0
  auth-trunc hmac(sha256) 0x60062d4d159455b5cf9a455877939aba2c18b1fd0cbd98b6bb4159eac1a292cf 128
  enc cbc(aes) 0x097098a65fa019f80cc2ffe3f6ef864d36dadd1d956982bc31985a51e4a5b811
  anti-replay context: seq 0x0, oseq 0x1, bitmap 0x00000000
  sel src 172.16.4.2/32 dst 172.16.5.2/32
src 172.16.5.2 dst 172.16.4.2
  proto esp spi 0xc0cc4f73 reqid 1 mode transport
  replay-window 32
  auth-trunc hmac(sha256) 0x95b1696c5f1614b99d0f2a402f9a6e5f980ee570d2fb8d640ef05d2ed84866a2 128
  enc cbc(aes) 0x009491f7d41377e813b422ba90428c5ea951176ec6e7e9ceff7c94b8e56947dc
  anti-replay context: seq 0x0, oseq 0x0, bitmap 0x00000000
  sel src 172.16.5.2/32 dst 172.16.4.2/32
src 172.16.4.2 dst 172.16.5.2
  proto esp spi 0xccc134e9 reqid 1 mode transport
  replay-window 0
  auth-trunc hmac(sha256) 0x89a664d7b3ddcbfa866cfab091e307ec091d7d302c60cfa0658a9736c6416cab 128
  enc cbc(aes) 0xe286c79903f4609d4f355a5a60ac9094ed14aa18cb88edd9aebc2c46732499bc
  anti-replay context: seq 0x0, oseq 0x1, bitmap 0x00000000
  sel src 172.16.4.2/32 dst 172.16.5.2/32
src 172.16.5.2 dst 172.16.4.2
  proto esp spi 0xcc9bed3a reqid 1 mode transport
  replay-window 32
  auth-trunc hmac(sha256) 0x7c9515efdfb8c8490c83f9cb2f3b2e8a41dfb0a5ee4a95253f2945c388e73d91 128
  enc cbc(aes) 0x91ad471d6c37ec2946b3325d2e6b30f3802e806bd33fa96f239855ac2683f8a1
  anti-replay context: seq 0x0, oseq 0x0, bitmap 0x00000000
  sel src 172.16.5.2/32 dst 172.16.4.2/32
```

Теперь вводим команду `ipsec statusall`

```
root@br-rtr:/home/user# ipsec statusall
Status of IKE charon daemon (strongSwan 5.5.1, Linux 5.4.0-110-generic, x86_64):
  uptime: 13 minutes, since Nov 11 21:19:28 2025
  malloc: sbrk 2433024, mmap 0, used 460368, free 1972656
  worker threads: 11 of 16 idle, 5/0/0/0 working, job queue: 0/0/0/0, scheduled: 3
  loaded plugins: charon aes rc2 sha2 sha1 md5 random nonce x509 revocation constraints pubkey pkcs1 pkcs7 pkcs8 pkcs12 pgp dnskey sshkey pem openssl fips-prf g
mp agent xcbc hmac gcm attr kernel-netlink resolve socket-default connmark stroke updown
Listening IP addresses:
  192.168.4.1
  172.16.5.2
  10.10.10.2
Connections:
  br-hq: 172.16.5.2...172.16.4.2 IKEv2
  br-hq: local: [172.16.5.2] uses pre-shared key authentication
  br-hq: remote: [172.16.4.2] uses pre-shared key authentication
  br-hq: child: dynamic === dynamic TRANSPORT
Security Associations (1 up, 0 connecting):
  br-hq[1]: ESTABLISHED 13 minutes ago, 172.16.5.2[172.16.5.2]...172.16.4.2[172.16.4.2]
  br-hq[1]: IKEv2 SPIs: 5c991eab350db6e6_1w_b6d7c1c262f9b4b_r, pre-shared key reauthentication in 2 hours
  br-hq[1]: IKE proposal: AES_CBC_256/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/MOVP_2048
  br-hq[1]: INSTALLED, TRANSPORT, reqid 1, ESP SPIs: ccc134e9_1 cc9bed3a_o
  br-hq[1]: AES_CBC_256/HMAC_SHA2_256_128, 0 bytes_i, 84 bytes_o (1 pkt, 788s ago), rekeying in 34 minutes
  br-hq[1]: 172.16.5.2/32 === 172.16.4.2/32
  br-hq[2]: INSTALLED, TRANSPORT, reqid 1, ESP SPIs: c6d8ccbe_1 c0cc4f73_o
  br-hq[2]: AES_CBC_256/HMAC_SHA2_256_128, 0 bytes_i, 84 bytes_o (1 pkt, 788s ago), rekeying in 29 minutes
  br-hq[2]: 172.16.5.2/32 === 172.16.4.2/32
  br-hq[3]: INSTALLED, TRANSPORT, reqid 1, ESP SPIs: c2bcebad_1 c304c6c9_o
  br-hq[3]: AES_CBC_256/HMAC_SHA2_256_128, 63448 bytes_i (721 pkts, 0s ago), 70168 bytes_o (801 pkts, 788s ago), rekeying in 34 minutes
  br-hq[3]: 172.16.5.2/32 === 172.16.4.2/32
```

Последим шагом, посмотрим пакеты, которые отправляются по gre туннелю. Для этого введем `tcpdump -ni eth0*`, после чего откроем второе окно терминала, нажав сочетание клавиш `alt+ctrl+f1` (`/f2/f3`) и во втором окне запустим `ping` по туннелю и вернемся в первое окно терминала. Вывод в `tcpdump` должен быть таким:

```

root@hq-rtr:/home/user# tcpdump -ni eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
21:35:15.434164 IP 172.16.4.2 > 172.16.5.2: ESP(spi=0xc2bcebad,seq=0x3b5), length 136
21:35:15.436808 IP 172.16.5.2 > 172.16.4.2: ESP(spi=0xc304c6c9,seq=0x3b5), length 136
21:35:16.435723 IP 172.16.4.2 > 172.16.5.2: ESP(spi=0xc2bcebad,seq=0x3b6), length 136
21:35:16.437692 IP 172.16.5.2 > 172.16.4.2: ESP(spi=0xc304c6c9,seq=0x3b6), length 136
21:35:17.437596 IP 172.16.4.2 > 172.16.5.2: ESP(spi=0xc2bcebad,seq=0x3b7), length 136
21:35:17.439617 IP 172.16.5.2 > 172.16.4.2: ESP(spi=0xc304c6c9,seq=0x3b7), length 136

```

*eth0 – имя интерфейса, который смотрит наружу.

Теперь, нужно проверить работу ospf, для этого на любом роутере вводим команду `vttysh`, после чего вводим `show ip ospf neighbor`, там должен отобразиться второй роутер как сосед

```

br-rtr# show ip ospf neighbor

```

Neighbor ID	Pri	State	Dead Time	Address	Interface	RXmtL	RqstL	DBsmL
192.168.2.1	1	Full/Backup	38.260s	10.10.10.1	gre1:10.10.10.2	0	0	0

4. Настройте межсетевой экран на маршрутизаторах HQ-RTR и BR-RTR на сеть в сторону ISP

Для настройки фаервола, будем использовать `ufw`, поскольку он будет самым простым вариантом. Обычно, он уже предустановлен в ОС, но, если же у вас его нету, скачайте и установите командой `apt-get install ufw`.

После этого начнем добавлять правила на HQ-RTR, поочередно выполняем команды:

`ufw allow in on eth0 proto tcp to any port 80 //HTTP`

`ufw allow in on eth0 proto tcp to any port 443 // HTTPS`

`ufw allow in on eth0 proto tcp to any port 53 // DNS TCP`

`ufw allow in on eth0 proto udp to any port 53 // DNS UDP`

`ufw allow in on eth0 proto udp to any port 123 // NTP`

```

root@hq-rtr:/home/user# ufw allow in on eth0 proto tcp to any port 80
Rules updated
Rules updated (v6)
root@hq-rtr:/home/user# ufw allow in on eth0 proto tcp to any port 443
Rules updated
Rules updated (v6)
root@hq-rtr:/home/user# ufw allow in on eth0 proto tcp to any port 53
Rules updated
Rules updated (v6)
root@hq-rtr:/home/user# ufw allow in on eth0 proto udp to any port 53
Rules updated
Rules updated (v6)
root@hq-rtr:/home/user# ufw allow in on eth0 proto udp to any port 123
Rules updated
Rules updated (v6)

```

Также, добавим правила

`ufw allow in on eth1 proto tcp to any port 2026 // ssh`

`ufw route allow in on eth1 out on eth0 // маршрутизация`

`ufw route allow in on eth0 out on eth1 // маршрутизация`

После чего включаем ufw командой `ufw enable` и смотрим добавленные правила командой `ufw status`

```

root@hq-rtr:/home/user# ufw enable
Firewall is active and enabled on system startup
root@hq-rtr:/home/user# ufw status
Status: active

To Action From
--
80/tcp on eth0 ALLOW Anywhere
443/tcp on eth0 ALLOW Anywhere
53/tcp on eth0 ALLOW Anywhere
53/udp on eth0 ALLOW Anywhere
123/udp on eth0 ALLOW Anywhere
2026/tcp on eth1 ALLOW Anywhere
80/tcp (v6) on eth0 ALLOW Anywhere (v6)
443/tcp (v6) on eth0 ALLOW Anywhere (v6)
53/tcp (v6) on eth0 ALLOW Anywhere (v6)
53/udp (v6) on eth0 ALLOW Anywhere (v6)
123/udp (v6) on eth0 ALLOW Anywhere (v6)
2026/tcp (v6) on eth1 ALLOW Anywhere (v6)

Anywhere on eth0 ALLOW FWD Anywhere on eth1
Anywhere on eth1 ALLOW FWD Anywhere on eth0
Anywhere (v6) on eth0 ALLOW FWD Anywhere (v6) on eth1
Anywhere (v6) on eth1 ALLOW FWD Anywhere (v6) on eth0

```

Все отлично, так что время настроить фаервол на BR-RTR. Вводим команды:

`sudo ufw allow in on eth1 proto tcp to any port 80 // http`

```
sudo ufw allow in on eth1 proto tcp to any port 443 // https
```

```
sudo ufw allow in on eth1 proto tcp to any port 53 // dns tcp
```

```
sudo ufw allow in on eth1 proto udp to any port 53 // dns udp
```

```
sudo ufw allow in on eth1 proto udp to any port 123 // ntp
```

```
sudo ufw allow in on eth0 proto tcp to any port 2026 // ssh
```

```
sudo ufw route allow in on eth1 out on eth0 // маршрутизация
```

```
sudo ufw route allow in on eth0 out on eth1 // маршрутизация
```

И проверяем правила

```
root@br-rtr:/home/user# ufw status
Status: active

To Action From
--
80/tcp on eth1 ALLOW Anywhere
443/tcp on eth1 ALLOW Anywhere
53/tcp on eth1 ALLOW Anywhere
53/udp on eth1 ALLOW Anywhere
123/udp on eth1 ALLOW Anywhere
2026/tcp on eth0 ALLOW Anywhere
80/tcp (v6) on eth1 ALLOW Anywhere (v6)
443/tcp (v6) on eth1 ALLOW Anywhere (v6)
53/tcp (v6) on eth1 ALLOW Anywhere (v6)
53/udp (v6) on eth1 ALLOW Anywhere (v6)
123/udp (v6) on eth1 ALLOW Anywhere (v6)
2026/tcp (v6) on eth0 ALLOW Anywhere (v6)

Anywhere on eth1 ALLOW FWD Anywhere on eth0
Anywhere on eth0 ALLOW FWD Anywhere on eth1
Anywhere (v6) on eth1 ALLOW FWD Anywhere (v6) on eth0
Anywhere (v6) on eth0 ALLOW FWD Anywhere (v6) on eth1
```

5. Настройте принт-сервер cups на сервере HQ-SRV

Для начала установим пакеты cups cups-pdf командой `apt-get install cups cups-pdf`, после чего сразу пропишем `systemctl enable --now cups`. После этого открываем конфиг `/etc/cups/cupsd.conf`, находим блоки `<Location />` и `<Location /admin>` и в каждый из них добавляем строку `Allow from all`

```

<Location />
    Allow from all
    Order allow,deny
</Location>

# Restrict access to the admin pages...
<Location /admin>
    Allow from all
    AuthType Default
    Require user @SYSTEM
    Order allow,deny
</Location>

```

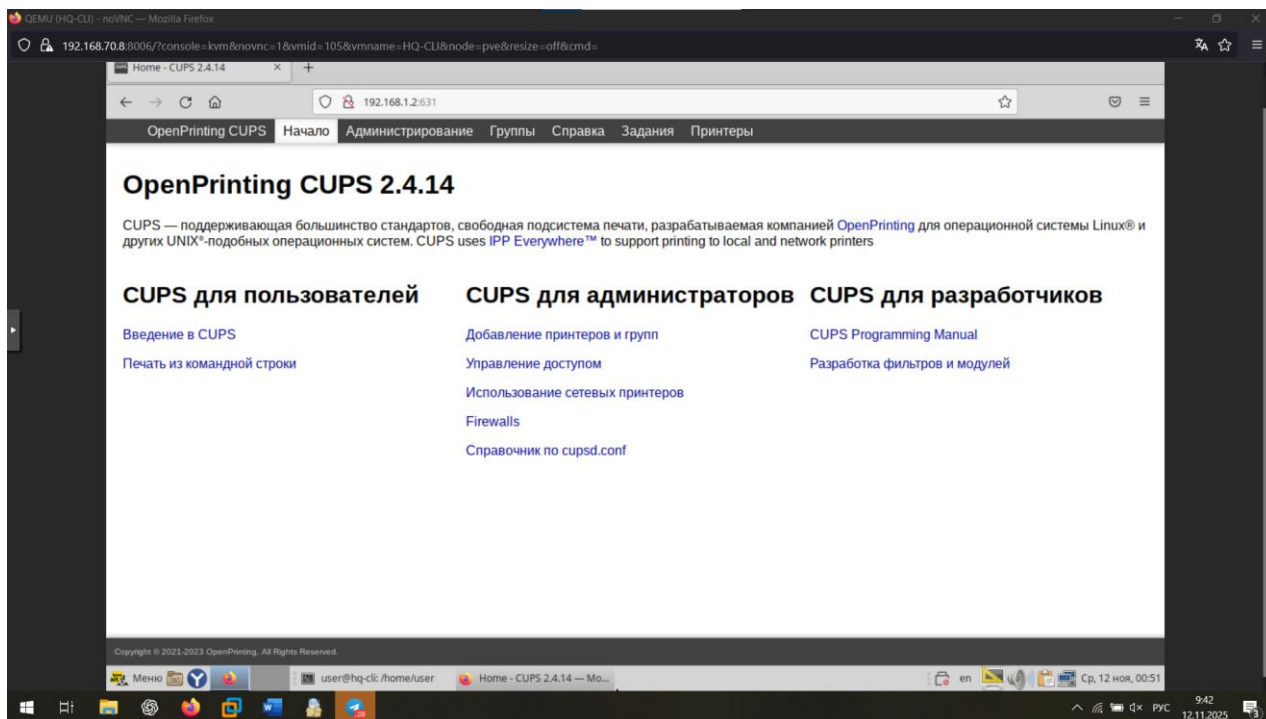
А также находим строку Listen и вводим значение 0.0.0.0:631

```

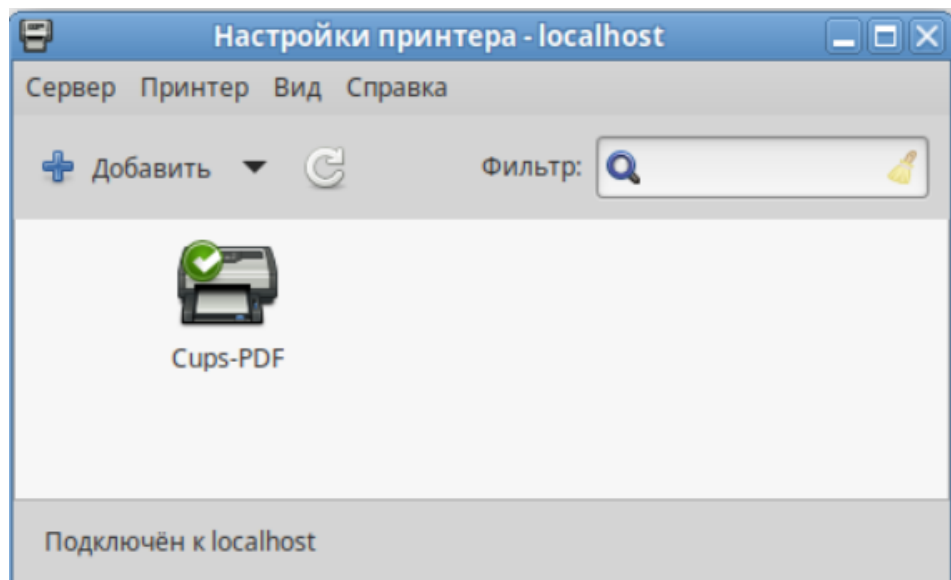
# Only listen for connections from localhost
Listen 0.0.0.0:631

```

После этого перезагружаем cups командой `systemctl restart cups` и переходим на HQ-CLI для проверки. Там в поисковую строку браузера вводим 192.168.1.2:631 и попадаем на страницу cups



Если страница открылась, значит все нормально. Теперь переходим в системные настройки и ищем там вкладку «Параметры печати», тут у нас должен был отображаться принтер cups-pdf



Если у вас он не отобразился автоматически, нажмите кнопку «Добавить» и введите URI «ipp://192.168.1.2:631/printers/Cups-PDF».

Теперь перейдем обратно на HQ-SRV. Выполним тестовую печать. Для этого вводим команду `echo "tEsT" | lp -d Cups-PDF`

```
[root@hq-srv print]# echo "debian is better" | lp -d Cups-PDF
request id is Cups-PDF-4 (0 file(s))
```

Проверим, сохранился ли файл в директории `/var/spool/cups-pdf/print/` командой `ls`

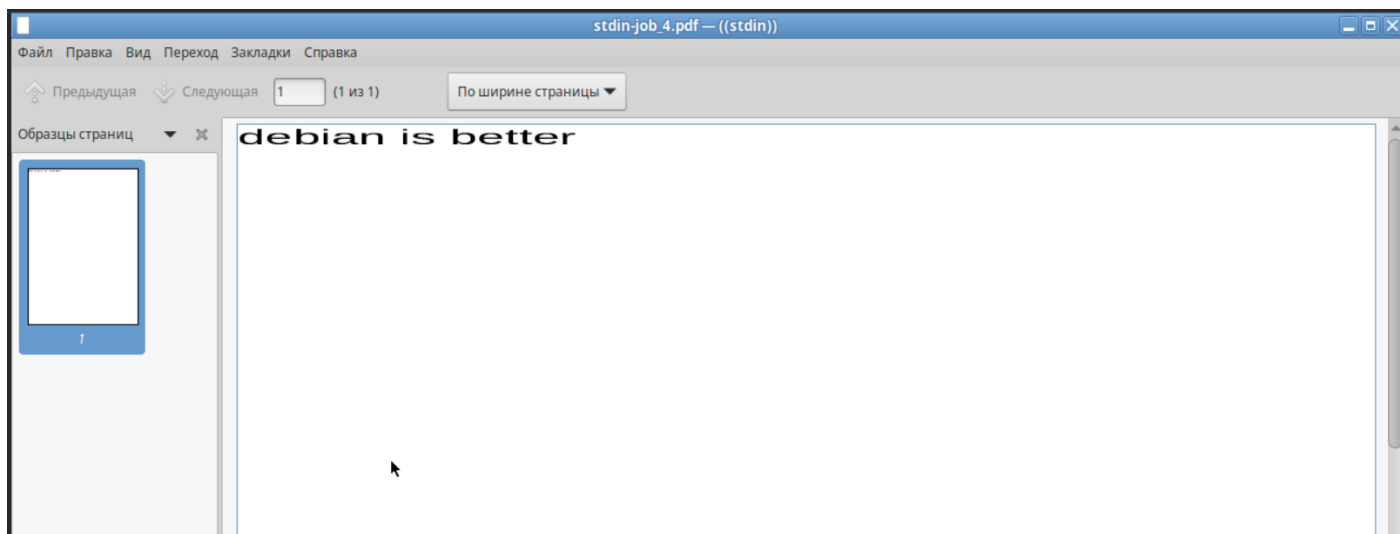
```
[root@hq-srv print]# ls /var/spool/cups-pdf/print/
stdin-job_1.pdf  stdin-job_1.txt  stdin-job_2.pdf
[root@hq-srv print]#
```

Так, файл вроде сохранился, но нужно ведь как-то проверить его содержимое? К сожалению, я не нашел рабочих способов сделать это с ОС без графической оболочки (по крайней мере `cat` у меня не сработал, может у вас работает, попробуйте 😊), поэтому передадим файл на HQ-CLI и посмотрим уже оттуда. Вначале, копируем файл `/var/spool/cups-pdf/print/stdin-job_1.pdf` в директорию `tmp` (для удобства), после чего переходим на HQ-CLI и вводим команду `scp -P 2026 sshuser@hq-srv:/tmp/stdin-job_1.pdf /home/user/Рабочий\ стол`

```
hq-cli ~ # scp -P 2026 sshuser@hq-srv:/tmp/stdin-job_3.pdf /home/user/Рабочий\ стол/
Authorized access only
sshuser@hq-srv's password:
```


*Перед этим может потребоваться выдать права `chmod 777` на `/tmp/stdin-job_4.pdf`.

Теперь, на рабочем столе у нас должен был появиться pdf-файл, откроем его, нажав два раза лкм



Можем увидеть, что файл в формате pdf создан и даже открывается(!). Теперь нам нужно распечатать что-нибудь с клиента. Для начала, на HQ-SRV вводим команду `cupctl --remote-any` и перезагружаем cups.

```
[root@hq-srv print]# cupctl --remote-any
[root@hq-srv print]# systemctl restart cups
```

Теперь с клиента пробуем распечатать, вводим команду `echo "test" | lp -h hq-srv -d Cups-PDF` и смотрим, появился ли файл на сервере.

*У меня почему-то для клиента создалась отдельная директория ANONYMOUS, хотя общий Out стоит в `/print/`

```
[root@hq-srv print]# ls -ah /var/spool/cups-pdf
.  ..  ANONYMOUS  SPOOL  print
```

```
[root@hq-srv print]# ls -ah /var/spool/cups-pdf/ANONYMOUS/
.  ..  stdin-job_5.pdf
```

Файл появился, значит все работает.

6. Реализуйте логирование при помощи rsyslog на устройствах HQ-RTR,

BR-RTR, BR-SRV

Сервер логирования у нас располагается на HQ-SRV. Инсталлируем на него пакеты rsyslog командой `apt-get install rsyslog`, после чего открываем конфиг `/etc/rsyslog.conf` и заполняем следующим образом:

```
include(file="/etc/rsyslog.d/*.conf" mode="optional")
module(load="imudp")
input(type="imudp" port = 514)

module(load="imtcp")
input(type="imtcp" port = 514)
```

Здесь:

- `Module(load="imudp")` – загружаем модуль для приема логов по UDP;
- `Input(type="imudp" port = 514)` – настраивает входной интерфейс для приема логов по udp на порту 514;
- `input(type="imtcp" port = 514)` – настраивает входной интерфейс для приема логов по tcp на порту 514.

После этого, создаем конфиг `/etc/rsyslog.d/10-remote.conf` и заполняем:

```
$template dyn, "/opt/log/%HOSTNAME%/%PROGRAMNAME%.log"
*.warn ?dyn
& stop
```

Здесь:

- `template dyn, "/opt/log/%HOSTNAME%/%PROGRAMNAME%.log"` – создать шаблон "dyn" для путей сохранения логов;
- `*.warn ?dyn` – применять шаблон "dyn" ко логам, уровень важности которых не ниже warn
- `stop` – Останавливать дальнейшую обработку после этого правила

Теперь настраиваем клиентов – BR-SRV, HQ-RTR, BR-RTR. На все устройства инсталлируем пакеты rsyslog-classic. Также, на всех устройствах открываем

конфигурационный файл `/etc/systemd/journald.conf` и раскомментируем строку `ForwardToSyslog=no` и поменяем ее значение на `yes`

```
ForwardToSyslog=yes_
```

*Главное, не делайте этого на HQ-SRV, поскольку тогда он будет клиентом самому себе.

Создаем конфигурационный файл `/etc/rsyslog.d/09-cli.conf` и вписываем строку «*.*@192.168.1.2:514»

```
*.* @192.168.1.2:514_
```

*Можно поставить @@, чтобы использовался протокол TCP, а не UDP.

После этого перезагружаем службу syslog командой `systemctl restart syslog`. Для проверки на любом из клиентов введем команду `logger -p user.err "TEST"`

```
[root@br-srv ~]# logger -p user.err "TEST"
[root@br-srv ~]#
```

после чего проверим директорию `/opt/log/` на HQ-SRV

```
[root@hq-srv print]# cat /opt/log/br-srv/root.log
Nov 12 14:18:09 br-srv root: TEST
[root@hq-srv print]# ls /opt/log/
br-srv  hq-srv
[root@hq-srv print]# ls /opt/log/br-srv/
root.log
[root@hq-srv print]# cat /opt/log/br-srv/root.log
Nov 12 14:18:09 br-srv root: TEST
[root@hq-srv print]#
```

Также, попробуем отправить сообщение уровня ниже `warning`, например, `info`. Для этого все на том же клиенте введем команду `logger -p user.info "TEST info"`

```
[root@br-srv ~]# logger -p user.info "TEST info"
[root@br-srv ~]#
```

И также смотрим директорию `/opt/log/br-srv`

```
[root@hq-srv print]# ls /opt/log/br-srv/  
root.log  
[root@hq-srv print]# cat /opt/log/br-srv/root.log  
Nov 12 14:18:09 br-srv root: TEST  
[root@hq-srv print]#
```

Ничего нового не появилось, значит фильтрация работает как надо.

Теперь нам нужно настроить ротацию логов. Для этого на HQ-SRV создаем конфиг `/etc/logrotate.d/remote.logs` и заполняем:

```
/opt/log/**/*.*.log {  
    weekly  
    rotate 4  
    compress  
    missingok  
    notifempty  
    size 10M  
    create 0640 root adm  
    sharedscripts  
    postrotate  
        systemctl reload rsyslog >/dev/null 2>&1 || true  
    endscript  
}
```

Здесь:

- `opt/log/**/*.*.log` - цель обработки, ротируются все файлы, входящие в поддиректории `/opt/log/**` и имеющие `.log` в конце;
- `weekly` - ротация раз в неделю;
- `rotate 4` - хранить 4 архивных копии;
- `compress` - сжимать старые логи (gzip);
- `missingok` - не ошибка если файл отсутствует;
- `notifempty` - не ротировать пустые файлы;
- `size 10M` - ротировать если размер достиг 10 МБ;
- `create 0640 root adm` - создавать новые файлы с правами 640, владелец root:adm;
- `sharedscripts` - выполнять скрипты один раз для всех логов;
- `postrotate` - скрипт после ротации:

- systemctl reload rsyslog - перезагрузка rsyslog для использования новых файлов.

Для проверки выполняем команду `logrotate -d /etc/logrotate.d/remote.logs`, смотрим

ВЫВОД:

```
[root@hq-srv print]# logrotate -d /etc/logrotate.d/remote.logs
WARNING: logrotate in debug mode does nothing except printing debug messages! Consider using verbose mode (-v) instead if this is not what you want.

reading config file /etc/logrotate.d/remote.logs
warning: 'size' overrides previously specified 'weekly'
Reading state from file: /var/lib/status
Allocating hash table for state file, size 64 entries
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Handling 1 logs

rotating pattern: /opt/log/**/*.*.log 10485760 bytes (4 rotations)
empty log files are not rotated, old logs are removed
considering log /opt/log/br-srv/root.log
Creating new state
Now: 2025-11-12 06:27
Last rotated at 2025-11-12 06:00
log does not need rotating (log size is below the 'size' threshold)
considering log /opt/log/hq-rtr/dhcpd.log
Creating new state
Now: 2025-11-12 06:27
Last rotated at 2025-11-12 06:00
log does not need rotating (log size is below the 'size' threshold)
considering log /opt/log/hq-srv/crond.log
Creating new state
Now: 2025-11-12 06:27
Last rotated at 2025-11-12 06:00
log does not need rotating (log size is below the 'size' threshold)
considering log /opt/log/hq-srv/rsyslogd.log
Creating new state
Now: 2025-11-12 06:27
Last rotated at 2025-11-12 06:00
log does not need rotating (log size is below the 'size' threshold)
not running postrotate script, since no logs were rotated
```

Все нормально, теперь пробуем принудительно ротировать наши логи, для этого используем команду `logrotate -f /etc/logrotate.d/remote-logs`

```
[root@hq-srv print]# logrotate -f /etc/logrotate.d/remote.logs
[root@hq-srv print]#
```

После этого смотрим директории логов клиентских машин командой `ls /opt/log/br-srv`

```
[root@hq-srv print]# ls /opt/log/br-srv/
root.log  root.log.1.xz
```

Можем увидеть, что у нас появился архив с логами root, значит ротация сработала как надо.

7. На сервере HQ-SRV реализуйте мониторинг устройств с помощью открытого программного обеспечения

Для реализации мониторинга будем использовать Grafana и Prometheus, поскольку на мой взгляд на данный момент это самое удобное, быстроработоспособное и вообще круто ПО.

Первым делом, нам требуется скачать и установить пакеты графаны и прометеуса. Для этого используем команду `apt-get install Grafana Prometheus Prometheus-node_exporter`, сразу после установки пропишем `systemctl enable --now Prometheus`, `systemctl enable --now grafana-server` и `systemctl enable --now prometheus`. После этого открываем yaml-файл конфигурации прометеуса по пути `/etc/prometheus/prometheus.yml`, находим в нем блок `scrape_configs` и меняем следующим образом:

```
scrape_configs:
  # The job name is added as a label `job=<job_name>` to a
  - job_name: 'node_exporter'

  # Override the global default and scrape targets from
  scrape_interval: 5s
  scrape_timeout: 5s

  # metrics_path defaults to '/metrics'
  # scheme defaults to 'http'.

  static_configs:
    - targets: ['192.168.1.2:9100', '192.168.4.2:9100']
```

После чего перезапускаем прометеус командой `systemctl restart prometheus`.

Настройка клиентов:

Устанавливаем пакет `apt-get install prometheus-node_exporter`, сразу пишем `systemctl enable --now prometheus-node_exporter`, ну и как бы все, можете еще проверить статус командой `systemctl status Prometheus-node_exporter.service`

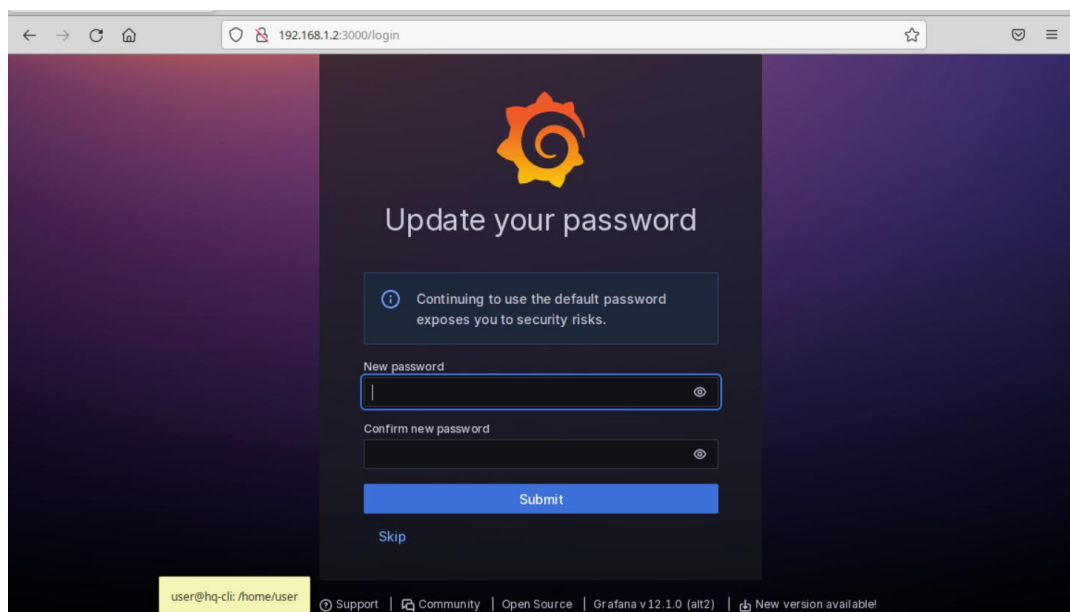
```

[root@br-srv ~]# systemctl status prometheus-node_exporter.service
prometheus-node_exporter.service - Prometheus Node Exporter Service
Loaded: loaded (/lib/systemd/system/prometheus-node_exporter.service; enabled; vendor preset: enabled)
Active: active (running) since Wed 2025-11-12 15:39:52 +05; 1min 57s ago
TriggeredBy: ● prometheus-node_exporter.socket
Docs: https://github.com/prometheus/node_exporter
Main PID: 5790 (node_exporter)
Tasks: 5 (limit: 2340)
Memory: 11.3M
CPU: 6.788s
CGroup: /system.slice/prometheus-node_exporter.service
        └─ 5790 /usr/bin/node_exporter --web.systemd-socket --collector.textf

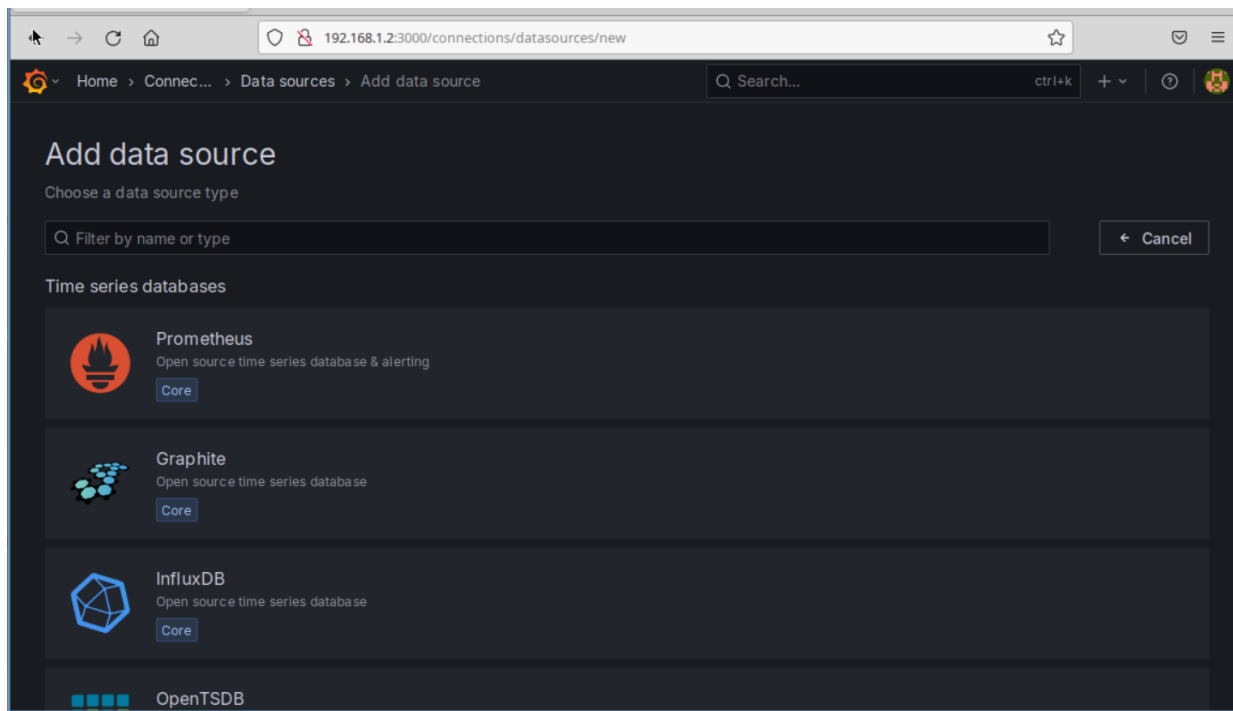
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.65
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.65
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.65
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.66
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.66
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.66
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.66
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.66
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.66
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.66
Nov 12 15:39:53 br-srv.au-team.irpo node_exporter[5790]: ts=2025-11-12T10:39:53.65
lines 1-22/22 (END)

```

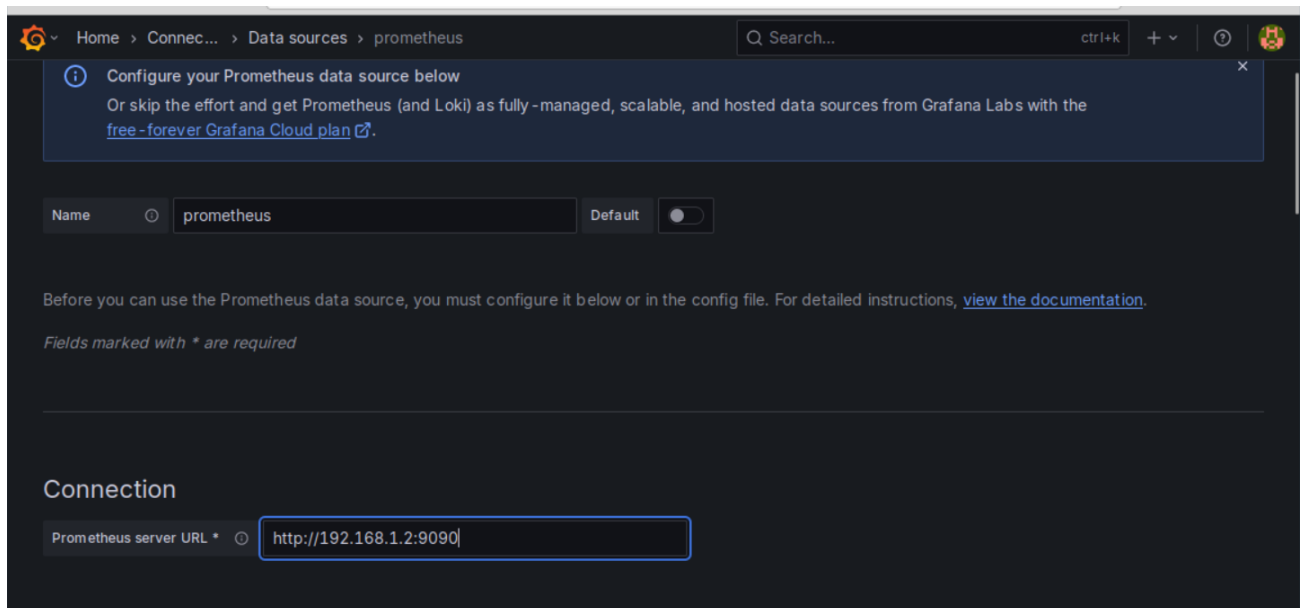
Сейчас можем заходить на HQ-CLI и в поисковой строке браузера вводить 192.168.1.2:3000 (3000 – дефолтный порт графаны). Мы попадем на страницу графаны, где нам нужно сначала войти по дефолтным данным admin admin, после чего нас сразу заставят сменить пароль



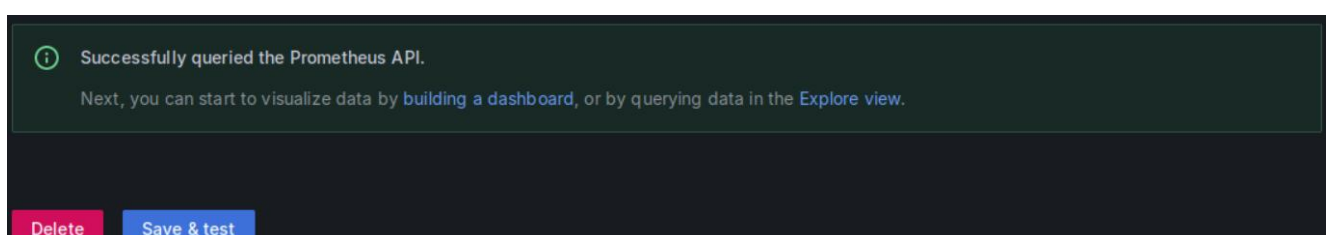
Когда у нас загрузится сайт, в левом верхнем углу нажимаем на значок графаны, в появившемся меню выбираем Data Sources > Add data source, у нас появится выбор источника метрик, выбираем Prometheus



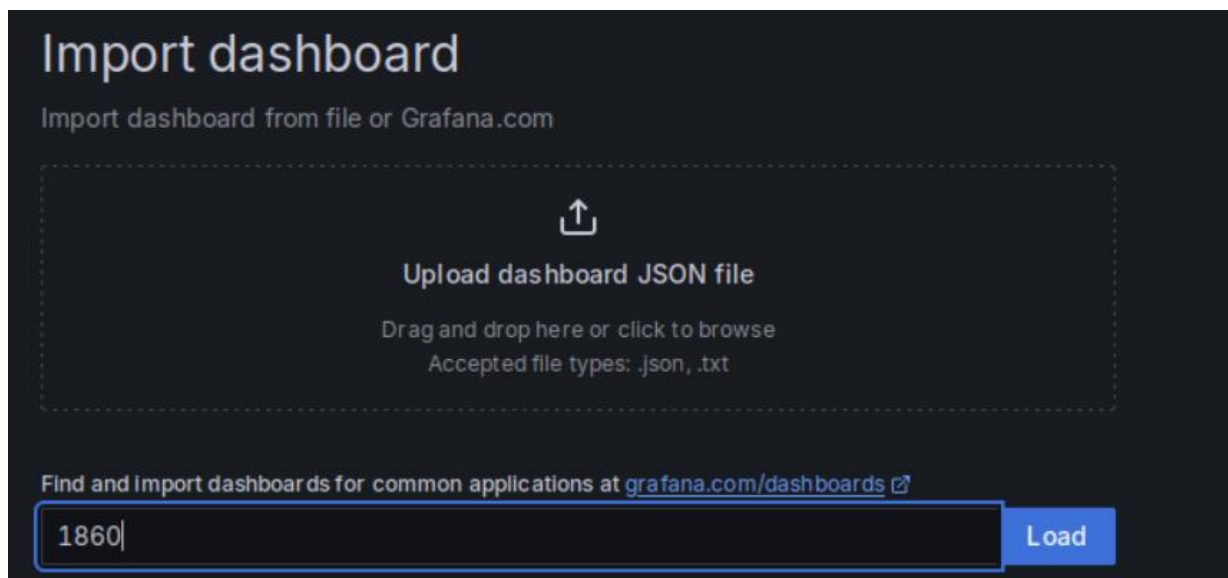
После этого в поле connection нам нужно будет ввести URI сервера Prometheus, вводим 192.168.1.2:9090



Листаем в низ страницы и жмем «Save&test» и у нас должны появиться зеленая плашка с надписью Successfully quired the Prometheus API, на этой плашке будет ссылка “building a dashboard”, жмем на нее



Нас перекинет на страницу, где можно будет выбрать дашборд, нажимаем “Import dashboard”, там вводим id дашборда 1860

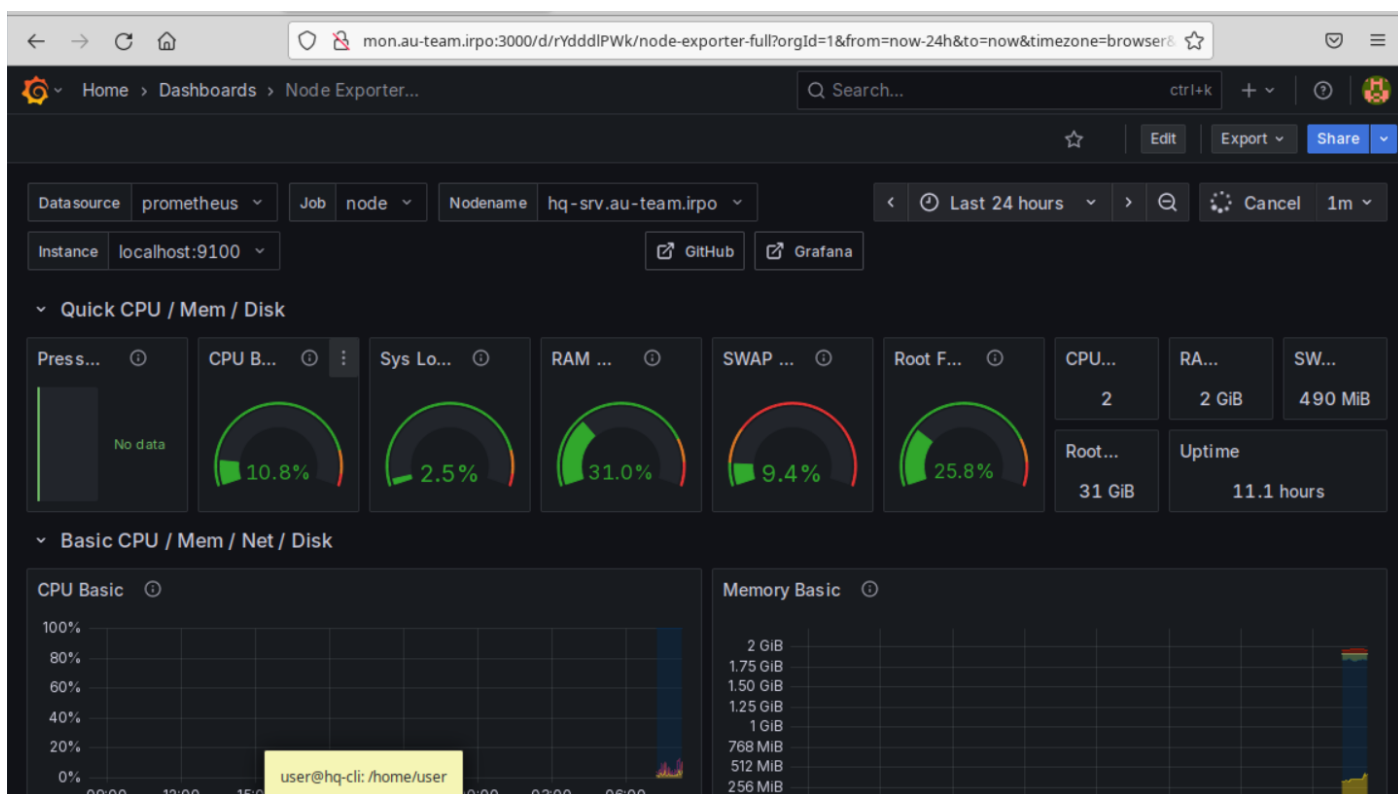


Пока у нас очень долго грузится страница дашборда, можно успеть настроить DNS так, чтобы графана открывалась по доменному имени `mon.au-team.irpo`. Для этого на HQ-SRV открываем конфигурацию прямой зоны ДНС по пути `/etc/bind/db.au-team.irpo` и добавляем одну запись:

```
mon    IN    A      192.168.1.2
```

```
mon    IN    A      192.168.1.2
```

Перезагрузим бинд командой `systemctl restart bind`, после чего с `hq-cli` попробуем войти на сайт по доменному имени, введя в поисковую строку браузера <http://mon.au-team.irpo:3000>



На этом скриншоте можем увидеть, что вход по доменному имени работает, а также, что метрики собираются отлично.

Осталось только разрешить доступ к нему исключительно из подсети HQ. Для этого на hq-srv вводим следующие правила iptables:

- `Iptables -A INPUT -p tcp --dport 3000 -s 192.168.1.0/27 -j ACCEPT`
- `Iptables -A INPUT -p tcp --dport 3000 -s 192.168.2.0/28 -j ACCEPT`
- `Iptables -A INPUT -p tcp --dport 3000 -j DROP`

```
[root@hq-srv print]# iptables -A INPUT -s 192.168.1.0/27 -p tcp --dport 3000 -j ACCEPT
[root@hq-srv print]# iptables -A INPUT -s 192.168.2.0/28 -p tcp --dport 3000 -j ACCEPT
[root@hq-srv print]# iptables -A INPUT -p tcp --dport 3000 -j DROP
```

На BR-SRV я выполнил команду `curl 192.168.1.2:3000` до и после внесения правил, ИТОГ:

```
[root@br-srv ~]# curl 192.168.1.2:3000
<a href="/login">Found</a>.
[root@br-srv ~]# curl 192.168.1.2:3000
```

А значит правила работают и задание завершено.

8. Реализуйте механизм инвентаризации машин HQ-SRV и HQ-CLI через Ansible на BR-SRV

*Для выполнения этого задания у вас обязательно должно быть выполнено задание номер 5 из второго модуля

Для начала создаем директорию поддиректорию pc-info в директории /etc/ansible/ командой `mkdir -p /etc/ansible/pc-info`

```
[root@br-srv ~]# mkdir -p /etc/ansible/pc-info
[root@br-srv ~]#
```

А также создаем директорию /etc/ansible/playbook командой `mkdir -p /etc/ansible/playbook`

```
[root@br-srv ~]# mkdir -p /etc/ansible/playbook/
```

Теперь копируем из образа Additional.iso файл `get_hostname_address.yml` в директорию `/etc/ansible/playbook` командой `cp`

`/mnt/add_cd/playbook/get_hostname_address.yml /etc/ansible/playbook/inv.yml`

```
[root@br-srv ~]# cp /mnt/add_cd/playbook/get_hostname_address.yml /etc/ansible/playbook/inv.yml
[root@br-srv ~]# ls /etc/ansible/playbook/
inv.yml
```

И, может это я тупой, а может нам положили какой-то кривой файл, который у меня никак не получилось нормально использовать, так что практически полностью меняем его по следующему подобию:

```
- name: получение данных с хоста
  hosts:
    - hq-cli
    - hq-srv
  become: false
  gather_facts: yes

  tasks:
    - name: get hostname and ip
      set_fact:
        report:
          hostname: "{{ ansible_hostname }}"
          ip_address: "{{ ansible_default_ipv4.address | default('N/A') }}"
    - name: save on br-srv
      delegate_to: localhost
      copy:
        dest: "/etc/ansible/pc-info/ {{ inventory_hostname }}.yml"
        content: |
          Hostname: {{ report.hostname }}
          IP_Address: {{ report.ipv4_address }}
        mode: '0644'
```

После этого, проверяем работоспособность плейбука командой `ansible-playbook /etc/ansible/playbook/inv.yml`

```
[root@br-srv ~]# ansible-playbook /etc/ansible/playbook/inv.yml

PLAY [получение данных с хоста] *****

TASK [Gathering Facts] *****
ok: [hq-cli]
ok: [hq-srv]

TASK [get hostname and ip] *****
ok: [hq-cli]
ok: [hq-srv]

TASK [save on br-srv] *****
ok: [hq-srv -> localhost]
changed: [hq-cli -> localhost]

PLAY RECAP *****
hq-cli      : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
hq-srv      : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Также проверим директорию pc-info командой `ls /etc/ansible/pc-info`

```
[root@br-srv ~]# ls /etc/ansible/pc-info/
' hq-cli.yml'  ' hq-srv.yml'
```

9. На HQ-SRV настройте программное обеспечение fail2ban для защиты ssh

Для начала установим пакет fail2ban командой `apt-get install fail2ban`, после чего копируем введем `systemctl enable --now fail2ban`. Теперь копируем `/etc/fail2ban/jail.conf` в `/etc/fail2ban/jail.local`, после чего открываем `jail.local` и в блок `[sshd]` (если нету, создайте) добавляем строки:

```
[sshd]
port      = 2026
logpath   = /var/log/auth/secure
enabled   = true
maxretry  = 3
findtime  = 600
bantime   = 60
filter    = sshd
```

Port = 2026 // указываем порт, на котором работает ssh;

Logpath = /var/log/auth/secure // указываем путь к файлу логов аутентификации;

***У вас может быть другой файл с логами, зависит от версии дистрибутива, будьте внимательны.**

enabled = true // включение fail2ban для этого сервиса;

maxretry = 3 // максимум 3 неудачные попытки входа;

findtime = 600 // за период 600 секунд (10 минут);

bantime = 60 / блокировка на 60 секунд (1 минута);

filter = sshd // использует фильтр для анализа SSH-логов.

После этого перезагружаем fail2ban командой `systemctl restart fail2ban` и смотрим статус командой `fail2ban-client status sshd`

```
[root@hq-srv ~]# fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 0
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
  |- Currently banned: 0
  |- Total banned: 0
  `-- Banned IP list:
```

Теперь подключаемся с любого другого устройства, подключаемся по ssh к HQ-SRV и 3 раза вводим неправильный пароль

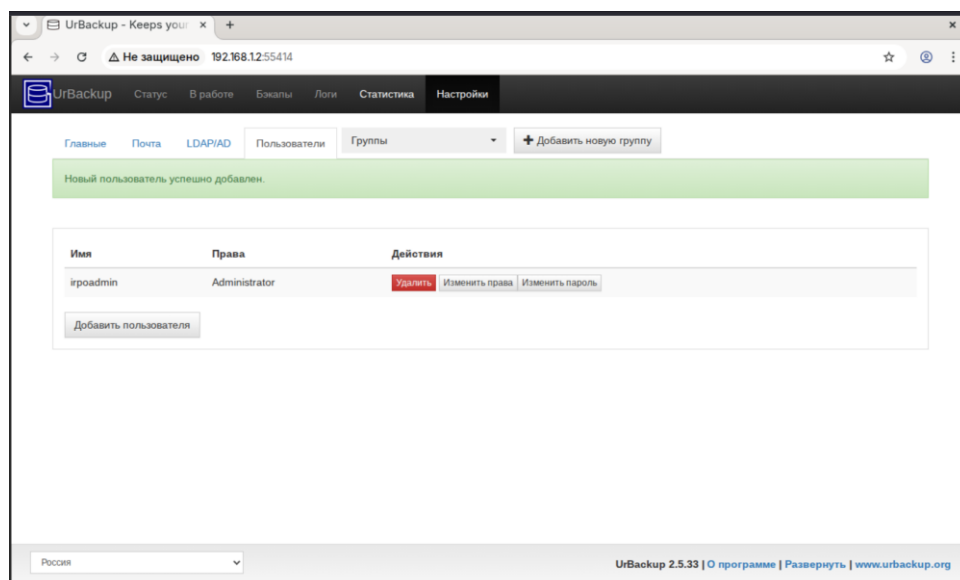
```
[root@br-rtr ~]# ssh -p 2026 sshuser@hq-srv
The authenticity of host '[hq-srv]:2026 ([192.168.1.2]:2026)' can't be established.
ED25519 key fingerprint is SHA256:GCuuZucL9mUGpIDXyJ+ZrIdTSk8at3UCUFlgpU9+rBk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[hq-srv]:2026' (ED25519) to the list of known hosts.
authorized access only
sshuser@hq-srv's password:
ssh: Permission denied, please try again.
sshuser@hq-srv's password:
ssh: Received disconnect from 192.168.1.2 port 2026:2: Too many authentication failures
Disconnected from 192.168.1.2 port 2026
[root@br-rtr ~]# ssh -p 2026 sshuser@hq-srv
ssh: connect to host hq-srv port 2026: Connection refused
[root@br-rtr ~]# ssh -p 2026 sshuser@hq-srv
ssh: connect to host hq-srv port 2026: Connection refused
```

10. Настройка резервного копирования директории сервера HQ-SRV.

Либо я вафля, либо Кибер бэкап, который нам рекомендуют по заданию надо покупать, либо же использовать пробную версию, но для этого нужна регистрация, так что этот вариант нам вообще не подходит. Также, имеется такая загвоздка, что

аналогов, способных удовлетворить все условия задания просто не существует, по крайней мере как бы я не искал – не нашел. Какой-то байт на покупку от составителей задания, видимо. Поэтому сюда я просто вставлю настройку другого бэкап сервера, который не соблюдает половину условий задания. Также, описано все будет очень плохо и вставлено мало скринов, поскольку этот отрывок я возьму из своего первого мануала, который я писал для себя.

1. Установим пакет `urbackup-server`
2. С HQ-CLI войдем в веб интерфейс по адресу `192.168.1.2:55414`
3. Переходим в настройки > пользователи и разрешения, создаем там организацию `igro` и пользователя админа с именем `igroadmin` и паролем `P@ssw0rd`



4. После чего также на HQ-CLI установим пакет `urbackup-client`
5. Создаем директорию для бекапов `mkdir /backup` с правами `chown urbackup:urbackup /backup`
6. После чего находим конфиг `/etc/default/urbackupclient` и меняем адрес сервера на `192.168.1.2` и перезапускаем службу `urbackupclient`
7. Добавляем HQ-CLI как storage node, для этого в веб интерфейсе заходим в настройки > хранилище > Добавить хранилище и вводим `host: hq-cli, path: /backup`. Ага! А тут таких кнопок нет, и че делать? Забить и не делать. Ладно, будем делать бекап при помощи другой утилиты, которая называется `borg`.

1. На HQ-CLI и HQ-SRV устанавливаем `borg`.

2. На кли создаем юзера `useradd -m irpoadmin` и ставим владельца на `/backup/irpoadmin`
3. После этого на срв создадим ключ для подключения по ssh без пароля `ssh-keygen -t ed25519` и копируем его на кли `ssh-copy-id -p 2026 irpoiadmin@hq-cli`(перед этим в конфиге `sshd` на кли добавим разрешенного `irpoadmin` в разрешенных юзеров)
4. После этого инициализируем хранилище на кли. Для этого на срв выполняем `borg init--encryption=repokey - -rsh "ssh -p 2026" irpoiadmin@hq-cli:/backup/hq-srv`
5. Создадим скрипт для резервного копирования `/usr/local/bin/backup_etc.sh` и вводим туда следующее:

```
#!/bin/bash
export BORG_REPO="irpoadmin@hq-cli:/backup/hq-srv"
export BORG_PASSPHRASE='P@ssw0rd'
borg create --rsh "ssh -p 2026" --stats --compression lz4 ::etc-$(date +%Y-%m-%d) /etc
borg prune --rsh "ssh -p 2026" -u --keep-daily=7 --keep-weekly=4 --keep-monthly=6
```

6. Если все норм, можем проверять, для этого сделаем файл исполняемы `chmod +x /usr/local/bin/backup_etc.sh` и `/usr/local/bin/backup_etc.sh`. Вывод должен быть примерно таким:

```
[root@hq-srv ~]# /usr/local/bin/backup_etc.sh
-----
Repository: ssh://irpoadmin@hq-cli/backup/hq-srv
Archive name: etc-2025-10-29
Archive fingerprint: fd2051c069fe6c6aa202870ed3a007b3231d0a91d4c1991ce0c9ec572e3ee6f6
Time (start): Wed, 2025-10-29 23:26:22
Time (end): Wed, 2025-10-29 23:27:04
Duration: 42.02 seconds
Number of files: 1941
Utilization of max. archive size: 0%
-----

```

	Original size	Compressed size	Deduplicated size
This archive:	40.56 MB	9.05 MB	9.02 MB
All archives:	40.56 MB	9.05 MB	9.18 MB

```
-----

```

	Unique chunks	Total chunks
Chunk index:	1824	1842

```
-----
[root@hq-srv ~]#
```